CST4714 | OPEN COURSE

OPERATING CLOUD DATABASES

PostgreSQL, MongoDB, and Reliable Data Systems

RELATION
rows + rules

DOCUMENT
shape + access

CLOUD
service + operations

A LAB-FIRST COURSE IN DATABASE REASONING AND OPERATIONS

ATILIO BARREDA

Second edition draft | CC BY-NC-SA 4.0

Contents

The page numbers below are generated from the same Word layout used for this PDF.

About This Book

Operating Cloud Databases is a beginner-friendly, lab-first textbook for students who need both a substantial relational/SQL review and an introduction to modern managed data systems. It develops one operating method across every topic: define the question, model the mechanism, run a controlled check, interpret the result, and state the tradeoff.

The book begins with relations, relational algebra, SQL, schema, and constraints. It then teaches safe change, transactions, access control, performance, and recovery in PostgreSQL and Supabase. This classroom volume contains Chapters 1-8. Later chapters of the developing textbook are intentionally outside this handoff.

This second edition adds mathematical notation, explicitly labeled code listings, editable conceptual diagrams, dated and redacted cloud interface figures, expanded relational review, and worked operational examples. The original edition remains preserved separately.

Who This Is For

The primary reader has written some SQL before but may not remember how relational operations explain a query. No prior cloud administration or MongoDB experience is assumed. Examples stay small enough to reason about, while the questions mirror real operational work: What does this query mean? What can fail? Who owns the control? Which result would change the decision? What remains uncertain?

How to Read a Chapter

Every chapter follows a stable learning pattern:

1. **Operating question:** the problem the chapter teaches you to reason about.
2. **Learning outcomes:** actions you should be able to perform and explain.
3. **Concepts and models:** durable ideas before product menus.
4. **Numbered code listings:** executable or inspectable examples with a named language and an explanation of the expected result.
5. **Worked example:** one decision carried from question through verification.
6. **Misconceptions:** plausible shortcuts that fail under closer inspection.
7. **Practice, retrieval, and transfer:** individual work that asks you to reuse the idea in a new situation.

The Recurring Case

Metro Support is a small synthetic service-request system with users, tickets, and ticket events. Its scale is intentionally modest. A beginner can inspect every row, while the same facts support joins, constraints, locks, access policies, indexes, backups, document models, aggregation, replication, integration, and incident communication.

The dataset is not presented as a production city system. It is a controlled environment for learning how an operator moves from a concrete question to a tested decision.

Cloud Interfaces and Change

Interface figures were captured from live free-tier teaching projects on August 25, 2026 and redacted before inclusion. They show what was observed, not a promise that a menu or plan will remain identical. Each screenshot is paired with the durable concept it demonstrates and a reminder to verify current official documentation before relying on a platform feature.

No student must purchase a textbook, subscription, certification, or database plan. Required cloud work has a local, static, or simulated route when an account, network, plan, or interface prevents access.

Accessibility and Formats

The canonical source uses semantic headings, descriptive links, text alternatives, language-labeled code blocks, and source equations that convert to Word equations and MathML. The same book is built as editable Word, PDF, standalone HTML, and EPUB so readers can select the format that works best with their device and assistive technology. Slide decks in the complete course package have structured text transcripts.

Weekly guides name the sections to read before each meeting. The chapters also offer optional practice for revision and transfer; those exercises are not extra graded submissions unless the instructor assigns them. A chapter's shell listing and its associated Python notebook teach related ideas in different languages. Use the stated environment and fixture rather than pasting shell code into a Python cell or assuming that two teaching datasets have identical rows.

Edition Status

This is the **unpublished Chapters 1-8 classroom volume**, prepared September 7, 2026. It is complete enough for substantive review and classroom piloting, but it is not a formally deposited or approved release. Version and review status appear in the source, exports, and fellowship release records so draft work cannot be mistaken for publication.

Part I: Relational Foundations

Database administration begins with meaning. Before tuning, backing up, or distributing a database, an operator must know what one row represents, which facts belong together, which rules make a state valid, and how a query transforms relations.

Chapters 1-3 establish the course's question-model-test cycle, rebuild relational algebra and SQL from first principles, and connect schemas and constraints to business meaning. The goal is not memorizing syntax in isolation. It is predicting a result, expressing the same operation precisely, and verifying whether the actual database state supports the claim.

1 How Database-Backed Applications Work

1.1 Opening Question

You tap **Submit** in an application. A moment later, a new support ticket, playlist, order, or game item appears on screen. What happened between the click and the stored result?

That question is the starting point for database administration. Before tuning, backing up, securing, or scaling a database, you need a useful mental model of the complete system around it.

We will follow one fictional service, Metro Support, from a resident's request to the reports used by staff. Keeping one case lets us see how an early decision has consequences later. An optional assignee affects a join. A status spelling affects a workload count. A second copy affects what a resident sees after an update. Database administration begins with understanding those connections, not with memorizing a dashboard.

1.2 Learning Outcomes

After this chapter, you can:

- distinguish data, a database, a database management system, and a managed cloud platform;
- trace a request from an application to stored data and back;
- explain how PostgreSQL relates to Supabase and how MongoDB relates to Atlas;
- describe the technical work performed by database administrators, backend developers, data engineers, and cloud support staff;
- recognize where schemas, queries, identities, networks, and storage can affect one request; and
- create a simple Markdown course file with GitHub's browser editor.

1.3 Begin With a Familiar Application Action

Suppose a resident submits a streetlight repair ticket. The application may send data resembling this request:

Listing 1. JSON

```
{
  "requester_id": 101,
  "category": "streetlight",
  "priority": "high",
  "description": "Lamp is dark at Harbor Avenue"
}
```

The browser does not usually write directly to a database. A common path is:

1. the browser or mobile client collects input;
2. an application server or API checks the request;
3. the application authenticates the user and determines what the user may do;
4. the application sends a SQL statement or MongoDB operation to a DBMS;
5. the DBMS checks types, constraints, permissions, and transaction state;
6. the DBMS reads or changes stored data; and

7. the result travels back through the application to the user.

A delay or error can occur at any stage. The database is important, but it is not the entire application.

The request body above is input, not proof of identity. A caller could change `requester_id` from 101 to somebody else's number. The server must establish who the caller is and either derive the requester ID from that identity or check that the caller is permitted to act for the named person. A valid JSON object and a valid database foreign key would not, by themselves, make that request authorized. We will separate those responsibilities carefully in Chapter 6.

1.3.1 Persistence Is Different From What the Screen Shows

The browser may hold unsaved text in memory. The API may hold a request while it is being processed. The DBMS manages the stored record. These are different states, even when they display the same words. Closing a browser tab should not delete a ticket that the service has already accepted into durable storage. Conversely, showing a friendly confirmation before a database write completes can promise more than the system has done.

A database also contains more than what one screen chooses to retrieve. Consider this small example, which we will use in the Week 1 lab:

Ticket	Status	Assigned agent
1001	open	201
1003	resolved	201
1004	new	none yet
1009	new	none yet

Suppose the staff screen first keeps active requests, where active means `new` or `open` in this small example. That leaves 1001, 1004, and 1009. It then matches each request to a known agent and retains only successful matches. Now only 1001 appears. Nothing in that sequence deleted either unassigned request. The screen has answered "which active requests already have a matching agent?" rather than "which requests still need attention?"

The appropriate repair is to preserve an active request even when its agent is missing, then label the missing assignment clearly. Adding a larger server, restoring a backup, or recreating the missing requests would not repair this query's meaning. This is why we review the relational model before studying performance and recovery. A fast, highly available system can still give the wrong answer.

1.3.2 A Confirmation Can Be Lost After a Write Succeeds

Now consider a different failure. The DBMS stores a new request, but the network connection breaks before the API's reply reaches the browser. The resident sees an error even though the ticket exists. Pressing Submit again might create a second ticket unless the application can recognize the repeated operation.

At this point we do not need a particular implementation. We need the right distinction: "the client did not receive success" is not identical to "the database did not make the change." Later chapters introduce

transactions, operation identifiers, and retries. Each is a way to make this uncertainty manageable rather than pretending it cannot occur.

1.4 Four Terms That Should Not Be Blurred Together

1.4.1 Data

Data represents facts or observations. In the ticket example, `101`, `streetlight`, and `high` are values. Their column or field names provide meaning.

1.4.2 Database

A **database** is an organized collection of data. It may contain tables, documents, indexes, views, constraints, and metadata.

1.4.3 Database Management System

A **database management system**, or **DBMS**, is the software that stores and retrieves data while coordinating users and programs. PostgreSQL and MongoDB are DBMSs. They provide query languages, access controls, indexes, transactions, concurrency behavior, and recovery mechanisms.

1.4.4 Managed Database Platform

A **managed platform** runs a DBMS on cloud infrastructure and adds management services. The platform may handle servers, storage hardware, software updates, monitoring, and parts of high availability. The customer still creates data models, queries, indexes, users, network rules, and application code.

"Managed" describes an allocation of work. It is not a guarantee that every application rule, query, permission, or recovery procedure is correct. If a provider replaces a failed disk, that addresses one infrastructure problem. It does not determine whether our analyst should be allowed to read residents' email addresses. A useful administrator can identify the particular control responsible for a result instead of treating the provider as one indivisible box.

Two comparisons will recur throughout this course:

DBMS	Managed platform used in class	Main representation
PostgreSQL	Supabase	relations, usually shown as tables
MongoDB	MongoDB Atlas	BSON documents, usually written in JSON-like syntax

Supabase is not a replacement name for PostgreSQL. It is a platform built around PostgreSQL and additional services. Atlas similarly operates MongoDB deployments and provides cloud management tools.

1.4.5 Why Database Systems Separate Logical and Physical Decisions

In the relational model, a user asks for facts by naming relations, attributes, and conditions rather than by naming disk locations. This distinction is part of *data independence*: programs should not need to change merely because the system changes how it stores or finds the same facts. Codd's 1970 paper made protection from representation changes a central motivation for relational databases. The original paper's IBM research

record provides historical context; the chapter explains the concept without requiring a paid article download.

For example, the question “which requests are still open?” remains the same if the DBMS adds an index. The index is a physical access choice, and the query can retain its logical meaning. If we instead rename a field or redefine what open means, we have changed an interface or a data contract. A consumer may need revision even though the files remain on the same disk. Chapter 4 studies how to manage that second kind of change.

This separation is an aim, not perfect insulation. Physical choices can change latency and resource cost, and those changes matter to an application. The important advantage is being able to discuss logical correctness and physical execution as related but different questions.

1.5 Read a Cloud Dashboard Without Treating It as the Database

Figure 1.1 shows a Supabase project overview. The screen combines several kinds of information: project health, database location, compute size, request counts, and shortcuts to other platform services. The **Primary Database** card refers to PostgreSQL; the surrounding dashboard belongs to the Supabase platform.

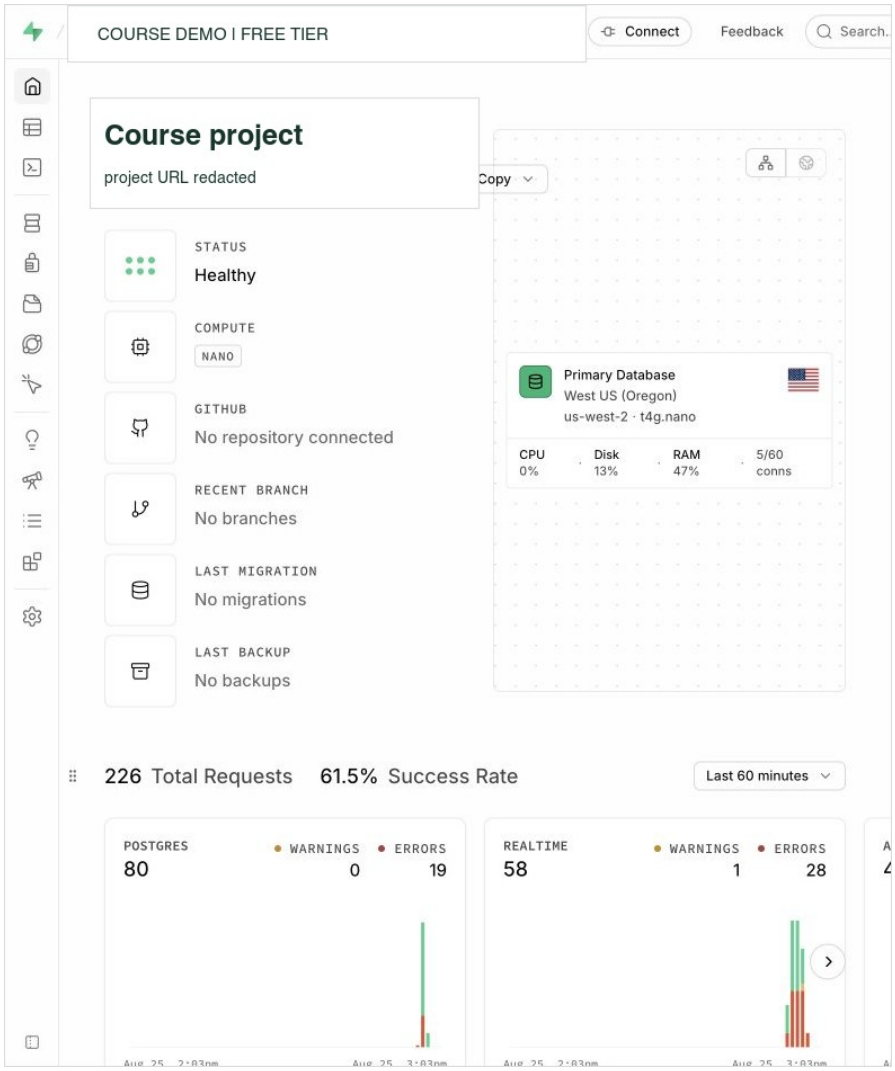

Figure 1.1: A redacted Supabase Free project overview. The database is one component inside a larger managed platform. Interface captured August 25, 2026.

Figure 1.2 shows an Atlas project overview. The project contains a MongoDB deployment and links to Data Explorer, network access, database users, backup, and other services. The yellow network notice matters because a running database can still be unreachable from the current client.

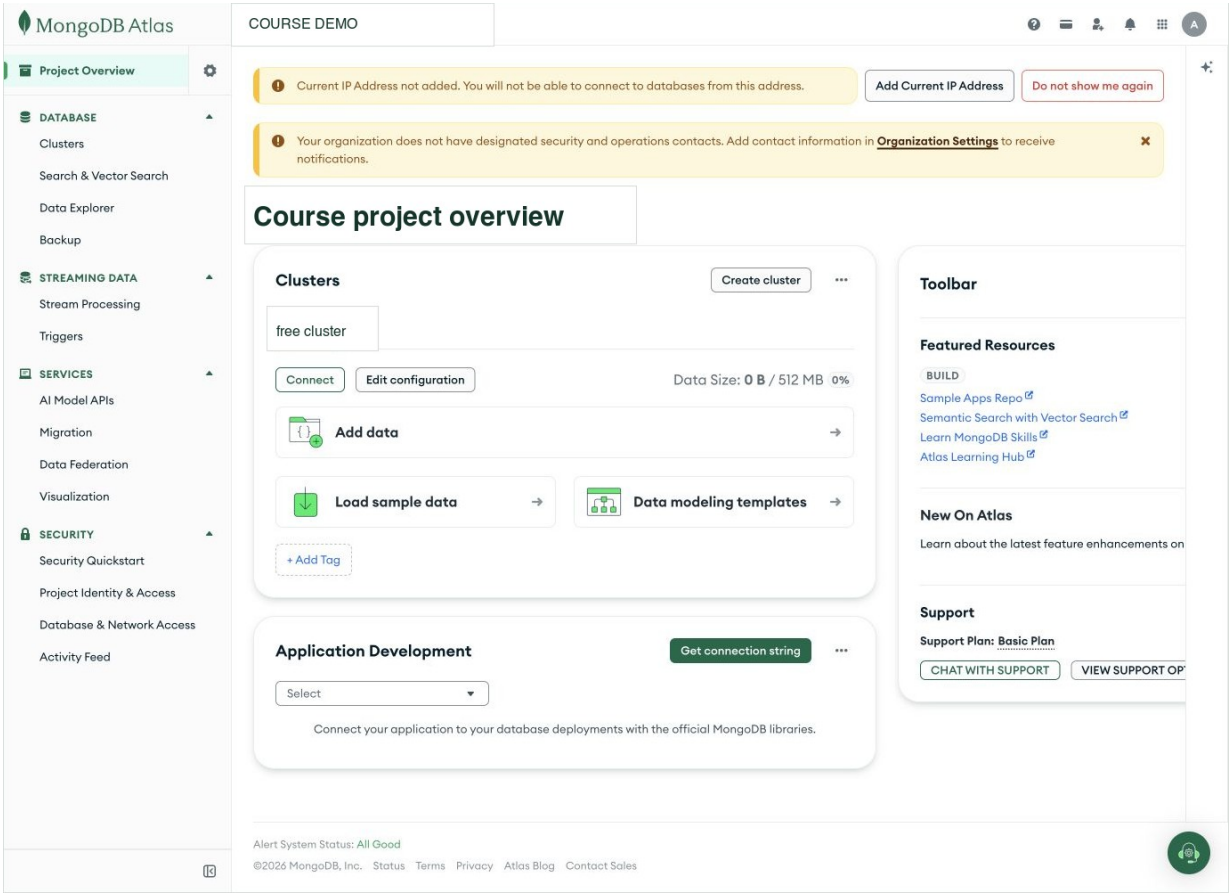

Figure 1.2: A redacted MongoDB Atlas project overview showing a free deployment and the surrounding platform controls. Interface captured August 25, 2026.

The labels and menus may change, but the underlying distinctions remain useful:

- **project or organization access** controls who can manage the cloud account;
- **database identity** controls who can connect to the DBMS;
- **network access** controls which clients can reach the service;
- **schema or document design** controls how data is represented; and
- **queries and indexes** control what work the database performs.

1.6 What Database Professionals Actually Do

Database work appears under several job titles. A database administrator may work beside backend developers, data engineers, site reliability engineers, security analysts, and cloud support staff. The boundaries differ by workplace, but the technical work commonly includes the following.

1.6.1 Model Data

Choose tables, documents, keys, relationships, types, and constraints that match the facts the application needs to store.

1.6.2 Manage Access

Create roles and users, grant only necessary privileges, protect credentials, and review who can read or change sensitive data.

1.6.3 Change Schemas and Data Safely

Plan migrations, preserve compatibility, use transactions where appropriate, and avoid losing or misinterpreting existing data.

1.6.4 Investigate Performance

Read query plans, measure expensive work, design indexes for real queries, and consider the cost of maintaining those indexes.

1.6.5 Coordinate Concurrent Work

Understand what happens when many sessions read and write at the same time. Diagnose blocking, deadlocks, and transaction behavior without guessing.

1.6.6 Prepare for Failure

Create backups or exports, rehearse restores, understand replication and failover, and decide how much data loss or downtime an application can tolerate.

1.6.7 Automate and Communicate

Use scripts, notebooks, version control, logs, and runbooks so recurring work is consistent and understandable to the next person.

This course practices all of these areas at an introductory level.

1.7 The Same Ticket in Two Database Models

A relational design might separate users and tickets into tables:

The SQL below illustrates definitions. You do not need to execute it in Week 1, and it is not the full Metro Support setup used in the later labs.

Listing 2. SQL

```sql
CREATE TABLE users (
    user_id      integer PRIMARY KEY,
    display_name text NOT NULL
);

CREATE TABLE tickets (
    ticket_id    integer PRIMARY KEY,
    requester_id integer NOT NULL REFERENCES users(user_id),
    category     text NOT NULL,
    priority     text NOT NULL,
    status       text NOT NULL
);
```

The foreign key states that each `requester_id` must identify an existing user. A query can join the two relations when it needs both ticket and requester data.

A document design might store a bounded requester summary inside a ticket:

Listing 3. JSON

```json
{
  "ticket_id": 1001,
  "requester": {
    "user_id": 101,
    "display_name": "Maya Chen"
  },
  "category": "streetlight",
  "priority": "high",
  "status": "open"
}
```

Neither shape is automatically better. The relational design gives the user one central row and joins when needed. The document design can read a ticket and its requester summary together but must decide what happens if the display name changes. Later chapters connect these choices to access patterns, consistency, and growth.

1.8 A Practical Troubleshooting Order

Consider the report: “The ticket page spins and then says it could not load.” Do not immediately rewrite a query or create an index. First locate the failing part of the request path.

Area	Useful first question	Example observation
Client	Did the browser send a request?	request is absent, pending, or returned an HTTP status
Application/API	Did the route run and accept the identity?	application log or authentication error
Network	Can this client reach the database endpoint?	DNS, timeout, or allow-list error
DBMS	Did the database receive work?	active session, database error, or lock wait
Query and data	Did the operation ask for too much or meet an unexpected data shape?	query text, plan, row count, or rejected value

The purpose of this table is not to produce paperwork. It prevents random changes to several layers at once.

1.9 Protect Credentials From the First Class

A password, connection string, API key, or private token can grant real access. Never place one in a public repository, shared slide, submitted notebook output, or chat message.

Use these habits throughout the course:

- enter secrets only when the program is running;
- use environment variables or the notebook's password prompt;
- keep `.env` files out of Git;
- remove secrets from outputs before sharing work; and
- rotate a credential immediately if it is exposed.

The course datasets use synthetic names and the reserved `example.test` domain, so they do not contain real student or customer records.

1.10 GitHub as the Course Code Notebook

GitHub stores versions of text files and notebooks. In this course, it can hold SQL, JSON, Markdown, and small scripts. You do not need to know the command line on the first day.

To create a Markdown file in the browser:

1. open your course repository;
2. choose **Add file** and **Create new file**;
3. enter a path such as `week_01/app_database_map.md`;
4. write in the editor and check the **Preview** tab; and
5. choose **Commit changes** with a short description.

Markdown uses simple punctuation for structure:

Listing 4. Markdown

```
# Music App Database Map

## Data the app stores

- users
- playlists
- songs

## Questions the database must answer

1. Which songs are in this playlist?
2. Which playlists belong to this user?
```

The weekly lab states the submission format for that activity.

1.11 Worked Example: Trace One Request

Action: An agent changes ticket 1001 from `open` to `in_progress`.

Possible request path:

1. the browser sends the ticket identifier and new status;
2. the API identifies the agent;
3. an authorization rule permits agents to update assigned tickets;
4. PostgreSQL begins a transaction;
5. a constraint checks that `in_progress` is an allowed status;
6. PostgreSQL updates the row and commits; and
7. the API returns the new ticket state.

This one action touches identity, authorization, a transaction, a constraint, stored data, and application behavior. The remaining course studies those pieces one at a time and then reconnects them.

1.12 In-Class Connection

Before the first meeting, read through **Four Terms That Should Not Be Blurred Together**, including the request-path examples. Before the second, review **Persistence Is Different From What the Screen Shows** and the opening relational-model sections of Chapter 2. In class, use the same missing-request case to explain a read path and a write path. The weekly page supplies the single lab submission; the questions below are study practice, not an additional report.

1.13 Chapter Summary

- A database is stored information; a DBMS is the software that manages it; a cloud platform operates the DBMS with additional services.
- PostgreSQL is the relational DBMS used by Supabase. MongoDB is the document DBMS used by Atlas.
- A user action travels through several layers before it becomes stored data.
- Database work includes modeling, access control, schema change, concurrency, performance, recovery, automation, and communication.
- SQL and document models organize related facts differently; the workload determines which tradeoffs matter.
- Credentials must never be committed or shared.

1.14 Check Your Understanding

1. Explain the difference between PostgreSQL and Supabase.
2. Explain the difference between MongoDB and Atlas.
3. Name three places a request can fail before the DBMS executes a query.
4. Why might a foreign key be useful in the ticket example?
5. What tradeoff appears when a user name is embedded in a ticket document?
6. Name four kinds of work performed by database professionals.

1.15 Further Reading

- PostgreSQL, “About PostgreSQL”
- Supabase architecture
- MongoDB Atlas documentation
- GitHub Skills, “Introduction to GitHub”
- O*NET, Database Administrators

2 Relational Operations Become Testable SQL

2.1 Operating Question

How can a query answer a real question while making it possible to check whether the answer is trustworthy?

A database can execute exactly the query we wrote while answering a different question from the one we intended. This is common when SQL has become a collection of remembered keywords rather than a model of how rows combine. We will rebuild that model, starting with a few visible facts and ending with queries whose behavior we can explain before and after execution. The aim is not speed at typing SQL. It is being able to find a mistake that the SQL parser cannot find.

2.2 Learning Outcomes

After this module, you can:

- explain rows, columns, keys, and relationships in a relational model;
- translate selection, projection, rename, product, join, union, and difference between relational-algebra notation and plain language;
- use `SELECT`, `WHERE`, `ORDER BY`, `JOIN`, `GROUP BY`, and aggregate functions;
- review subqueries, common table expressions, and safe `INSERT`, `UPDATE`, and `DELETE` patterns;
- reason about `NULL` without treating it as zero or an empty string;
- distinguish the written order of a query from its logical processing order; and
- verify a query with counts, boundary cases, and comparison queries.

2.3 A Relation Represents One Kind of Fact

2.3.1 The Instance Used in This Chapter

The worked SQL uses the complete Metro Support baseline: **8 users, 12 tickets, and 21 ticket events**. Load the supplied schema and seed script into an isolated PostgreSQL practice database. A new web-editor tab may use a new database session, so the examples use names such as `metro_support.tickets` rather than relying on an earlier `SET search_path` command.

The SQL review notebook starts with a smaller 4-user/6-ticket/9-event instance to make the first demonstrations easy to inspect. Its final **Complete Week 2 Lab Dataset** section switches to the full baseline used here and in both labs. A different instance can produce different answers to the same correct query. Always identify which data you are reasoning about before comparing counts.

The relational model organizes data into relations commonly presented as tables. A row represents one occurrence, and each column represents one attribute of that kind of occurrence. A key identifies a row. A foreign key records a relationship by referring to a key in another table.

Metro Support separates:

- `users`: one row per person or staff account;
- `tickets`: one row per support request; and
- `ticket_events`: one row per recorded event in a ticket's history.

This separation avoids repeating a user's name and email in every ticket event. It also creates work: a query must join related rows when the answer needs facts from more than one table.

2.3.2 Schema, Instance, Domain, and Key

A *schema* describes the structure: for example, tickets have an identifier, requester, status, and opening time. An *instance* is the collection of rows that exists at a particular moment. The instance changes when a ticket arrives; the schema usually does not. Mixing up the two leads to rules that happen to fit today's data but do not describe tomorrow's valid data.

A *domain* describes the permitted kind of value for an attribute. Some domain information comes from a type such as integer or timestamp, and some comes from a rule such as the set of allowed statuses. A ticket identifier and an agent identifier may both use integers without meaning the same thing. Their names, keys, and relationships give them different roles.

A key is not simply a convenient-looking column. It must distinguish permitted rows. A person's current display name is a poor identifier if two people can share it or one person can change it. An assigned `user_id` allows relationships to remain stable through a name change. Chapter 3 will separate the mathematical idea of a candidate key from the particular key chosen as the SQL primary key.

2.4 Relational Algebra Gives Us a Reasoning Vocabulary

Relational algebra describes how one or more input relations become an output relation. The notation is less important than the habit of decomposing a question into operations.

Assume T is the tickets relation and U is the users relation. Let φ and θ stand for predicates: expressions that evaluate to true or false for a tuple or tuple pair.

Operation	Conventional notation	Plain-language job
selection	$\sigma_{\varphi}(T)$	keep tuples from T that satisfy predicate φ
projection	$\pi_{a_1,\ldots,a_k}(T)$	keep attributes a_1 through a_k
rename	$\rho_{S}(T)$	give relation T the name S
Cartesian product	$T \times U$	pair every tuple in T with every tuple in U
theta join	$T \bowtie_{\theta} U$	pair tuples that satisfy relationship predicate θ
union	$R \cup S$	include tuples that occur in either compatible relation
intersection	$R \cap S$	include tuples that occur in both compatible relations

Operation	Conventional notation	Plain-language job
difference	$R - S$	include tuples in R but not in compatible relation S

The subscripts carry the details of an operation. In $\sigma_{\varphi}(T)$, φ is the row condition. In $\pi_{a_1,\ldots,a_k}(T)$, the subscript is the attribute list. The letters R, S, T, and U name relations; they are not SQL keywords.

Classical projection selects attributes. SQL's expression list can also compute new values, which is often called generalized projection. Likewise, grouping and outer joins are useful extensions beyond the small classical algebra listed here. Knowing the boundary prevents us from forcing every SQL feature into a notation that was introduced for a simpler mathematical model.

Relational idea	Common SQL expression
selection	`WHERE`
projection	the expression list after `SELECT`
rename	`AS`
Cartesian product	`CROSS JOIN`
theta/equi-join	`JOIN ... ON`
union	`UNION`
intersection	`INTERSECT`
difference	`EXCEPT`

Union, intersection, and difference require *union-compatible* inputs: the same number of attributes with corresponding domains that can be compared. SQL adds practical details such as column names, data-type conversion, duplicate handling, and `NULL` semantics.

A theta join can be understood as a selection applied to a Cartesian product:

$$T \bowtie_{\theta} U = \sigma_{\theta}(T \times U)$$

This identity explains why a missing join predicate is dangerous. Without θ, the system retains the full product rather than only meaningful pairs.

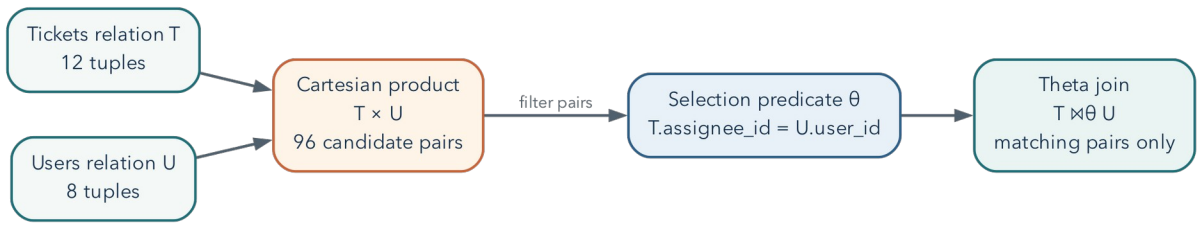

Figure 2.1: A theta join can be reasoned about as a Cartesian product followed by a selection predicate.

2.4.1 Example Translation

Question: Which high-priority tickets are still active, and what are their subjects?

Reasoning:

1. select tickets where priority is high and status is active;
2. project the ticket identifier, subject, and status; and
3. sort for presentation, which is useful SQL behavior but not a core relational- algebra operation.

Let A be the set of active status values:

$$A = \{\text{'new'}, \text{'open'}, \text{'in\_progress'}\}.$$

Define the predicate φ and then apply selection followed by projection:

$$\varphi = (\text{priority} = \text{'high'}) \wedge (\text{status} \in A)$$

$$Q = \pi_{\text{ticket\_id}, \text{subject}, \text{status}}\left(\sigma_{\varphi}(\text{tickets})\right)$$

Listing 5. SQL

```sql
SELECT ticket_id, subject, status
FROM metro_support.tickets
WHERE priority = 'high'
  AND status IN ('new', 'open', 'in_progress')
ORDER BY ticket_id;
```

2.4.2 Algebra and SQL Are Related, Not Identical

In the full baseline, the query above returns ticket IDs **1001 and 1011**. Ticket 1003 is urgent but already resolved, so it does not belong. Ticket 1005 is high priority but resolved, so it does not belong either. The excluded examples are as important as the included ones: they test that both parts of the question have survived the translation.

Classical relational algebra uses sets: duplicate tuples do not occur, and ordering is not part of a relation. SQL normally uses *bag* or multiset semantics, so duplicate result rows can occur unless a key, grouping operation, `DISTINCT`, or set operator changes that. SQL also includes `NULL` and three-valued logic, which require reasoning beyond classical algebra.

This difference explains two important habits: always state the expected result grain, and never assume row order without `ORDER BY`.

2.5 Start With a Precise Question

"Show tickets" is vague. "List open or in-progress high-priority tickets, newest first, with the assigned agent's name" is testable. It identifies:

- the unit: one ticket per result row;
- the filter: selected statuses and high priority;
- the order: newest first;
- the relationship: ticket to assigned user; and

- the desired attributes.

Before writing SQL, state the expected grain: what does one result row mean? Unexpected duplicates often reveal that the query changed grain.

2.6 Select, Filter, and Sort

Listing 6. SQL

```sql
SELECT ticket_id, subject, status, opened_at
FROM metro_support.tickets
WHERE status IN ('open', 'in_progress')
ORDER BY opened_at DESC;
```

`SELECT` names the output expressions. `FROM` identifies the source. `WHERE` removes rows that do not meet the predicate. `ORDER BY` controls presentation. Without `ORDER BY`, row order is not guaranteed, even when repeated runs appear consistent.

When two tickets share an opening time, that one-column sort does not determine their relative order. Add `ticket_id` as a tie-breaker if repeatable ordering matters: `ORDER BY opened_at DESC, ticket_id`. A stable order is particularly important when an application retrieves one page at a time.

2.6.1 Parentheses Preserve the Question

Suppose the question changes to active tickets with **high or urgent** priority. Write the two priority alternatives together:

Listing 7. SQL

```sql
SELECT ticket_id, priority, status
FROM metro_support.tickets
WHERE priority IN ('high', 'urgent')
  AND status IN ('new', 'open', 'in_progress')
ORDER BY ticket_id;
```

Without the grouping, `priority = 'high' OR priority = 'urgent' AND ...` means high priority *regardless of status*, or urgent priority with the additional status condition. SQL evaluates `AND` more tightly than `OR`. On the baseline, that mistake admits resolved high-priority tickets 1005 and 1008. The statement is legal SQL; the error is in the logical claim we encoded.

Use parentheses even when you know the precedence rule if they make that claim easier for the next reader to see. Choosing a compact expression is useful only when it preserves both meaning and readability.

Avoid `SELECT *` in durable work. Explicit columns document intent, reduce unnecessary transfer, and make a query less vulnerable to later schema changes.

2.7 SQL's Logical Order

SQL is written in a human-readable order but reasoned about approximately as:

1. `FROM` and `JOIN` build the source rows.
2. `WHERE` filters individual rows.
3. `GROUP BY` forms groups.
4. aggregate calculations summarize each group.

5. `HAVING` filters groups.
6. `SELECT` produces output expressions.
7. `DISTINCT`, if requested, removes duplicate output rows.
8. `ORDER BY` sorts the result.
9. `LIMIT` restricts the returned rows.

This explains why a `SELECT` alias is often unavailable in `WHERE`: the filter is logically evaluated before the output alias exists.

This is a reasoning order, not a literal instruction to the storage engine to build every intermediate table. An optimizer may push a safe filter earlier, choose a different join algorithm, or avoid sorting by reading an index in order. Those choices must preserve the query's semantics. Chapters 7 and 11 return to this distinction between the question and its physical execution.

2.8 Join Related Facts

Listing 8. SQL

```sql
SELECT
    t.ticket_id,
    t.subject,
    u.display_name AS assignee_name
FROM metro_support.tickets AS t
LEFT JOIN metro_support.users AS u
    ON u.user_id = t.assignee_id
ORDER BY t.ticket_id;
```

An `INNER JOIN` returns only matching pairs. A `LEFT JOIN` preserves every row from the left side and supplies `NULL` for missing right-side values. Because two tickets are unassigned, an inner join would silently remove them. The join type is therefore a statement about the question, not just syntax.

2.8.1 A Join Is a Filtered Product

A Cartesian product pairs every row in one relation with every row in another. With 12 tickets and 8 users, the product has 96 pairs. An equi-join keeps only pairs whose key values match. Thinking of a join as product plus selection makes a missing join condition easier to recognize.

Formally, if $|T|$ denotes the number of tuples in T, then:

$$|T \times U| = |T||U|.$$

For the class data, $|T| = 12$ and $|U| = 8$, so $|T \times U| = 96$. A selective join predicate should discard most of those candidate pairs.

2.8.2 Diagnose Duplicate Rows

Joining one ticket to many events changes the grain from one row per ticket to one row per matching event:

Listing 9. SQL

```sql
SELECT t.ticket_id, t.subject, e.event_type, e.event_at
FROM metro_support.tickets AS t
JOIN metro_support.ticket_events AS e
    ON e.ticket_id = t.ticket_id
WHERE t.ticket_id = 1003
ORDER BY e.event_at;
```

Three rows for ticket 1003 are correct because it has three events. The repeated ticket identifier refers to distinct events, not three separate tickets.

`DISTINCT` removes only rows identical across the selected expressions. If event type or time differs, those three rows remain distinct. If we project only the ticket ID, `DISTINCT` can deliberately answer "which tickets had a matching event?" But it does not turn an event-level sum into a ticket-level sum. We must choose the unit of analysis before choosing the aggregate, especially when the result could influence workload or service decisions.

2.9 `NULL` Means Missing or Inapplicable

`NULL` is not zero, blank text, or a normal value. Ordinary equality comparisons with `NULL` produce unknown rather than true. In a `SELECT` query, compare these two filter clauses:

Listing 10. SQL

```sql
-- Incorrect: this does not find unassigned tickets.
WHERE assignee_id = NULL

-- Correct.
WHERE assignee_id IS NULL
```

Three-valued logic includes true, false, and unknown. This matters for filters, constraints, joins, and aggregates. `count(*)` counts rows. `count(assignee_id)` counts only rows where that expression is not null.

The essential truth table for `AND` is:

p	q	$p \wedge q$
TRUE	TRUE	TRUE
TRUE	UNKNOWN	UNKNOWN
FALSE	UNKNOWN	FALSE
UNKNOWN	UNKNOWN	UNKNOWN

A `WHERE` clause keeps only rows for which its predicate is `TRUE`. Both `FALSE` and `UNKNOWN` are filtered out.

2.9.1 Why `NOT IN` Can Surprise You

The assigned-agent column contains nulls. Consequently, asking for users whose ID is `NOT IN (SELECT assignee_id FROM metro_support.tickets)` is not the same as asking whether no ticket has that user's ID.

A comparison against the null member is unknown, and negating unknown does not make it true. This can eliminate every otherwise unmatched user from the result.

An existence test states the intended question more directly:

Listing 11. SQL

```sql
SELECT u.user_id, u.display_name
FROM metro_support.users AS u
WHERE NOT EXISTS (
    SELECT 1
    FROM metro_support.tickets AS t
    WHERE t.assignee_id = u.user_id
)
ORDER BY u.user_id;
```

For each candidate user, the inner query asks whether a matching assignment exists. The selected constant `1` has no business meaning; `EXISTS` cares whether there is a row. Null assignments simply do not match an ordinary user ID. This query includes residents and other non-agent accounts because it asks about **all users**. Add a role condition only if the intended population is narrower.

2.10 Group and Aggregate

Listing 12. SQL

```sql
SELECT
    category,
    count(*) AS ticket_count,
    count(closed_at) AS closed_timestamp_count
FROM metro_support.tickets
GROUP BY category
ORDER BY ticket_count DESC, category;
```

After grouping, every selected expression must either identify the group or summarize it. The result grain is now one row per category.

Use `HAVING` for a condition on a group:

Listing 13. SQL

```sql
SELECT category, count(*) AS ticket_count
FROM metro_support.tickets
GROUP BY category
HAVING count(*) >= 3;
```

2.11 Worked Example: Staff Workload Including Zero Counts

Question: For every agent or supervisor, how many active tickets are assigned?

First define “active” as `new`, `open`, or `in_progress`. The result should have one row per staff member in those two roles, including supervisor Elena Garcia, who currently has none. Analysts and residents are outside this report’s stated population. This explicit definition matters: the baseline’s two agents both have active work, so an agents-only example would not show the zero-count case.

Listing 14. SQL

```sql
SELECT
    u.user_id,
    u.display_name,
    count(t.ticket_id) AS active_ticket_count
FROM metro_support.users AS u
LEFT JOIN metro_support.tickets AS t
    ON t.assignee_id = u.user_id
   AND t.status IN ('new', 'open', 'in_progress')
WHERE u.role IN ('agent', 'supervisor')
GROUP BY u.user_id, u.display_name
ORDER BY active_ticket_count DESC, u.display_name;
```

Why is the ticket-status condition in `ON` rather than `WHERE`? A `WHERE` condition on `t.status` would remove the null-extended row for an agent with no matching active ticket, making the left join behave like an inner join for this purpose.

Why `count(t.ticket_id)` instead of `count(*)`? The left join produces one row for an agent with no match, but `t.ticket_id` is null in that row. Counting the ticket identifier correctly yields zero.

The expected result is Priya Shah with 3, Noah Williams with 2, and Elena Garcia with 0. Those counts sum to 5, while the baseline has 7 active tickets. The two unassigned requests, 1004 and 1009, are not owned by any staff member and therefore do not appear in this *staff* grouping. Nothing was lost from the database. A complete operational dashboard could present the five assigned requests and a separate two-request unassigned queue.

This is a useful limit on the phrase “preserves missing rows.” A left join preserves the chosen left population, not every record in every participating table. In the earlier query, the left population was tickets. Here it is staff. Changing which side we preserve changes the question.

2.12 Subqueries and CTEs Name Intermediate Relations

A table subquery produces a result used as an input relation. A scalar subquery produces one value and must not return multiple rows. An `EXISTS` subquery answers a yes/no existence question. A common table expression, or CTE, gives an intermediate result a name. These are related tools, not interchangeable syntax.

Listing 15. SQL

```sql
WITH active_tickets AS (
    SELECT ticket_id, assignee_id, priority
    FROM metro_support.tickets
    WHERE status IN ('new', 'open', 'in_progress')
),
agent_counts AS (
    SELECT assignee_id, count(*) AS active_count
    FROM active_tickets
    WHERE assignee_id IS NOT NULL
    GROUP BY assignee_id
)
SELECT u.display_name, a.active_count
FROM agent_counts AS a
JOIN metro_support.users AS u
    ON u.user_id = a.assignee_id
ORDER BY a.active_count DESC, u.display_name;
```

Read each CTE as a named step in relational reasoning. A CTE can improve clarity, but it is not automatically faster. PostgreSQL’s optimizer and version determine how it is planned.

The CTE example above starts from assigned active tickets. It therefore returns Priya and Noah, but not Elena. That is intentional for this particular query; it is not equivalent to the preceding all-staff report. To include zero-workload staff, start with the staff relation and left join the CTE counts, using `COALESCE(active_count, 0)` for an absent group. Naming an intermediate result improves readability, but it does not excuse checking who is included.

Set operators express union and difference when inputs have compatible columns:

Listing 16. SQL

```sql
-- Users who requested a ticket but are not assigned to any ticket.
SELECT requester_id AS user_id
FROM metro_support.tickets
EXCEPT
SELECT assignee_id
FROM metro_support.tickets
WHERE assignee_id IS NOT NULL;
```

`UNION` removes duplicate rows. `UNION ALL` preserves them and avoids duplicate- elimination work when set semantics are not required.

2.13 Review Safe Data Changes

Queries read state; data-manipulation statements change it.

Use the following insertion as a rehearsal. The transaction rolls back the new event, so rerunning the example does not accumulate records or collide with the same event key. Begin from the baseline, where event 5099 is unused.

Listing 17. SQL

```sql
BEGIN;
INSERT INTO metro_support.ticket_events (
    event_id, ticket_id, actor_id, event_type, old_status, new_status, note, event_at
) VALUES (
    5099, 1001, 201, 'note_added', 'open', 'open',
    'Replacement lamp requested', now()
)
RETURNING event_id, ticket_id, event_type;
ROLLBACK;
```

Before an `UPDATE` or `DELETE`, write the same predicate as a `SELECT` and inspect the target keys:

Listing 18. SQL

```sql
SELECT ticket_id, priority
FROM metro_support.tickets
WHERE category = 'streetlight'
  AND priority = 'medium';

BEGIN;

UPDATE metro_support.tickets
SET priority = 'high'
WHERE category = 'streetlight'
  AND priority = 'medium'
RETURNING ticket_id, priority;

ROLLBACK;
```

`RETURNING` shows affected rows. A transaction creates a decision point. Neither replaces a correct predicate or independent verification.

Use the same discipline for deletion:

This deletion example is a predicate-and-transaction template. If you just ran the rolled-back insertion above, event 5099 does not exist, so it correctly deletes zero rows. To observe one deletion, create that disposable event inside this same transaction before the `DELETE`. Never change the predicate to a broad one merely to make the affected-row count nonzero.

Listing 19. SQL

```sql
BEGIN;

DELETE FROM metro_support.ticket_events
WHERE event_id = 5099
RETURNING event_id, ticket_id;

-- Verify the intended row and then choose COMMIT or ROLLBACK.
ROLLBACK;
```

Never practice broad destructive statements in a shared or production database.

2.14 Verification Is a Second Query or Reasoning Path

Use at least one of these methods:

- **Known total:** compare grouped counts with the total number of source rows.
- **Boundary case:** inspect an unassigned ticket, a null value, or a user with no match.
- **Simpler query:** verify one result category with a direct filtered count.
- **Uniqueness check:** compare total rows with distinct keys at the expected grain.
- **Manual sample:** trace one identifier across source tables.

For the workload query, verify Priya Shah directly:

Listing 20. SQL

```sql
SELECT ticket_id, status
FROM metro_support.tickets
WHERE assignee_id = 201
  AND status IN ('new', 'open', 'in_progress');
```

The verification query is intentionally less abstract. Independent reasoning is more valuable than repeating the same logic in a different format.

2.15 Common Misconceptions

2.15.1 “No rows means the query worked”

No rows could mean no data matches, the filter is wrong, a join removed records, or the setup failed. Verify source counts and test one known identifier.

2.15.2 "A join connects tables automatically"

SQL joins use the condition you write. A missing or incorrect condition can create a Cartesian product or pair unrelated records.

2.15.3 "`DISTINCT` fixes duplicates"

`DISTINCT` removes identical output rows. It does not fix an incorrect grain or relationship and may hide a modeling or join error.

2.16 Practice

For the assigned individual work, follow the Week 2 page. The first lab applies the filters and null reasoning to three questions. The second repairs a dashboard, counts staff workload, and rehearses an update. The additional exercise below is study or extension practice, not another required submission.

First express the required operations in plain language or relational-algebra notation. Then write a query that returns one row per neighborhood with:

- the number of resident users;
- the number of tickets they requested; and
- the most recent ticket opening time.

Before running it, predict which joins are required and whether a left join is necessary. After running it, verify one neighborhood with a simpler query.

2.17 Retrieval and Transfer

1. How do selection and projection differ?
2. Why is a join related to a Cartesian product?
3. What does one row in a query result represent?
4. Why can a left join return more rows than the left table?
5. What is the difference between `WHERE` and `HAVING`?
6. Why does `column = NULL` not work as expected?
7. When does `UNION ALL` express the intended result better than `UNION`?
8. What should you do before running an `UPDATE` or `DELETE`?
9. A report suddenly doubles its row count after an events table is joined. What should you inspect before adding `DISTINCT`?

2.18 Further Reading

- PostgreSQL `SELECT`
- PostgreSQL table expressions and joins
- PostgreSQL aggregate functions
- *Database Design - 2nd Edition*, Chapter 8

3 A Schema Protects Meaning

3.1 Operating Question

Which invalid states should the database refuse, even when an application has a bug or a user imports a bad file?

Imagine receiving three CSV files that produce the right report today. That does not yet tell us whether tomorrow's import can contain two records for the same ticket, an impossible date, or a request assigned to a person who does not exist. A schema turns selected assumptions into rules the DBMS can enforce. Choosing those rules requires understanding the facts, not simply selecting a type from a menu.

3.2 Learning Outcomes

After this module, you can:

- use PostgreSQL schemas as namespaces and permission boundaries;
- choose basic data types, primary keys, and foreign keys;
- explain how dependencies and normalization reduce inconsistent copies of facts;
- apply `NOT NULL`, `UNIQUE`, `CHECK`, and referential actions intentionally;
- distinguish an integrity constraint from an index; and
- inspect database metadata instead of relying on memory.

3.3 "Schema" Has Two Related Meanings

A database schema can mean the overall structure of data: tables, columns, relationships, constraints, and indexes. In PostgreSQL, a *schema* is also a named namespace inside a database. `metro_support.tickets` identifies the `tickets` table in the `metro_support` namespace.

Namespaces help:

- separate application objects from extensions or shared utilities;
- avoid name collisions;
- organize permissions; and
- make object ownership explicit.

PostgreSQL resolves unqualified names through `search_path`. In durable scripts, schema-qualified names reduce ambiguity:

Listing 21. SQL

```
SELECT ticket_id, status
FROM metro_support.tickets;
```

3.4 Dependencies Explain Why We Separate Tables

Suppose a ticket table repeats the requester's current name and email on every row. Maya has three tickets. When her email changes, one row is updated and two are not. The database now offers conflicting answers to the question "what is Maya's current email?" The problem is not that repetition is visually untidy. The same fact has several independently writable representations.

A *functional dependency* expresses a rule about which attributes determine others. Under our account model, one `user_id` determines one current display name and email:

$$\text{user\_id} \rightarrow \{\text{display\_name}, \text{email}\}.$$

This means that two valid rows with the same user ID cannot disagree about those attributes. It is a rule about every permitted instance, not a pattern proved by today's eight rows. If every ticket in a tiny sample happens to have a different category, that coincidence does not make category a valid ticket identifier.

A *superkey* determines the entire row. A *candidate key* is a minimal superkey: removing any attribute from it would lose that property. We choose one candidate as the primary key. Additional business identities may be protected with unique constraints. The word minimal concerns the included attributes, not whether the numeric values are small.

3.4.1 Insertion, Update, and Deletion Anomalies

Repeating current account details in a ticket table creates several problems. An update can change one copy and leave others behind. Inserting an account may require inventing a ticket merely to have somewhere to store the account's attributes. Deleting the person's last ticket may delete the only remaining record of the account. These are update, insertion, and deletion anomalies.

We separate `users` from `tickets` so each current account fact has one authoritative location. Tickets store the identifying reference. We can join the relations when a report needs both. That join has a computational cost, but it also reconstructs a combined view without making every combined view a new independently writable copy.

3.4.2 A Brief Normalization Review

First normal form is conventionally taught as using one value from the declared domain in each attribute rather than repeating groups such as `event1`, `event2`, and `event3`. A variable-length history should not require adding another column every time an event occurs. PostgreSQL can legitimately store arrays or `jsonb`; their availability does not decide whether a nested value is the best way to represent a particular relationship.

Second normal form addresses partial dependencies on a composite candidate key. For a table keyed by `(ticket_id, tag_id)`, a tag's current label depends on `tag_id`, not on the whole assignment pair. Store the label in a tag relation rather than repeating it for every ticket that uses the tag.

Third normal form further limits dependencies that let non-key facts determine other non-key facts. In the repeated-requester example, `ticket_id` determines `requester_id`, and requester identity determines the current email. Keeping the email in `users` avoids storing that dependency repeatedly in `tickets`. The formal 3NF condition is: for every nontrivial dependency $X \rightarrow A$, either X is a superkey or A belongs to a

candidate key. This qualification matters when a relation has several overlapping candidate keys. Boyce-Codd normal form uses the stricter requirement that each nontrivial determinant be a superkey.

For this course, the practical goal is to recognize where a current fact has multiple writable copies and explain a sensible decomposition. Decompositions should be *lossless*: joining the pieces through the intended keys must recover the original facts without inventing combinations. In Metro Support, each ticket's non-null requester points to one uniquely identified user, so joining that reference retrieves one requester record per ticket.

Normalization does not forbid every purposeful copy. A ticket may need the address reported **at the time of submission**, even after a resident moves. That historical address is a different fact from the account's current address. A document summary used for faster reads may also copy current data, provided we identify its owner and update policy. Chapters 10 and 14 revisit those deliberate choices. A label such as "denormalized" is not an explanation by itself.

3.5 Data Types Express Allowed Representation

A data type is the first integrity decision. Choose a type that represents the domain and supports required operations.

Need	Common PostgreSQL type	Reasoning
whole-number identifier	`integer` or `bigint`	numeric identity, efficient equality
arbitrary text	`text`	no invented length limit
exact money/measurement	`numeric(p,s)`	decimal precision rather than binary approximation
yes/no state	`boolean`	avoids multiple spellings
instant in time	`timestamptz`	stores an instant and converts display zones
calendar date	`date`	no implied time of day
structured flexible attribute	`jsonb`	queryable JSON when a relational column is not appropriate

Do not choose a type only because the current sample fits. Ask what values should exist and what operations must be correct.

An instant and a calendar label are different facts. `timestamptz` lets us compare the instant at which two events occurred even if clients display different time zones. It does not retain the user's original zone name. If the application must schedule "9 a.m. in New York" on future dates, keep the relevant zone or scheduling rule too. A ZIP code may look numeric but is often better stored as text because leading zeroes are significant and arithmetic on the value has no meaning.

3.6 Keys Identify and Connect Facts

A **primary key** uniquely identifies a row and is not null. A **foreign key** requires a value to match a key in another table, unless the foreign-key column is allowed to be null.

The following reduced definition illustrates those rules. The complete baseline already has a `tickets` table, so do not execute this `CREATE TABLE` again after loading it.

Listing 22. SQL

```
CREATE TABLE metro_support.tickets (
    ticket_id integer PRIMARY KEY,
    requester_id integer NOT NULL
        REFERENCES metro_support.users(user_id),
    assignee_id integer
        REFERENCES metro_support.users(user_id)
);
```

The nullable `assignee_id` represents a real state: a ticket may be unassigned. The non-null `requester_id` states that every ticket must have a known requester.

The assignee foreign key establishes that a user exists, not that the user has role `agent`. Assigning resident 101 would satisfy this particular reference. That is a useful example of a valid database state that may still violate a business rule we have not encoded. A richer design could separate eligible staff into a referenced relation. Application authorization and Chapter 6's permissions address other parts of the problem; none should be assumed from the foreign key alone.

3.6.1 Referential Actions Are Business Decisions

What should happen if a user row is deleted?

- `RESTRICT` or `NO ACTION`: refuse deletion while dependent rows exist.
- `CASCADE`: delete or update dependent rows automatically.
- `SET NULL`: preserve the dependent row but remove the reference.

There is no universally correct action. Cascading a requester deletion into all historical tickets would probably destroy important records. An anonymization workflow or restricted deletion may be safer.

3.7 Constraints Reject Invalid States

3.7.1 `NOT NULL`

Use when the fact must exist at row creation. Do not mark a field optional only because imports are inconvenient.

3.7.2 `UNIQUE`

Use when duplicate values would violate identity or a business rule:

Listing 23. SQL

```
-- The supplied baseline already declares email NOT NULL UNIQUE.
-- Inspect the existing rule instead of adding a second copy.
SELECT conname, pg_get_constraintdef(oid) AS definition
FROM pg_constraint
WHERE conrelid = 'metro_support.users'::regclass
  AND contype = 'u';
```

By default, PostgreSQL uniqueness does not treat two nulls as the same value. The baseline avoids that issue for email by also requiring a value. If a different design needs null to participate in uniqueness as one shared

missing value, PostgreSQL 15 and later support `UNIQUE NULLS NOT DISTINCT`. Choose the intended meaning rather than assuming every SQL system makes the same choice.

3.7.3 `CHECK`

Use a Boolean rule about one row:

Listing 24. SQL

```sql
ALTER TABLE metro_support.tickets
ADD CONSTRAINT tickets_priority_allowed
CHECK (priority IN ('low', 'medium', 'high', 'urgent'));
```

Listing 25. SQL

```sql
-- This equivalent date rule already exists in the supplied baseline.
-- The statement illustrates its definition; do not add a redundant copy.
ALTER TABLE metro_support.tickets
ADD CONSTRAINT tickets_close_after_open
CHECK (closed_at IS NULL OR closed_at >= opened_at);
```

Constraints should have meaningful names. An error mentioning `tickets_priority_allowed` is more useful than one mentioning a generated name.

A `CHECK` rejects false, but accepts true **or unknown**. Thus `CHECK (score >= 0)` does not prohibit a null score. Add `NOT NULL` if the value itself is required. This differs from `WHERE`, which retains only true. The same three-valued logic has different consequences at a filter and at a constraint boundary.

3.7.4 Constraints Are Not a Complete Workflow Engine

A row-level `CHECK` can keep `closed_at` after `opened_at`. It cannot easily prove that every status transition followed a multi-row workflow or external approval. Use the database for durable invariants and choose application or procedural logic for process rules that cross rows, time, or services.

PostgreSQL does not support cross-table queries inside a `CHECK` as a general integrity mechanism. A function that happens to look at another table does not make the dependency safely maintained when that other table changes. Use foreign keys or another explicitly coordinated mechanism for cross-row requirements. The PostgreSQL constraints reference documents these null and cross-row distinctions.

3.8 An Index Is an Access Structure, Not the Rule Itself

An index stores an organized path to rows. It can accelerate matching, joining, sorting, or uniqueness checks. It also consumes storage and adds work to inserts, updates, deletes, backups, and maintenance.

Listing 26. SQL

```sql
CREATE INDEX tickets_status_opened_idx
ON metro_support.tickets (status, opened_at DESC);
```

This index may support a workload that filters status and requests recent rows. It is not automatically useful for every query involving either column. Column order, selectivity, table size, and the query plan matter. Week 7 develops the facts and measurements needed to decide.

A primary key or `UNIQUE` constraint normally creates a supporting unique index, but the concepts differ. The constraint states a rule. The index is a mechanism PostgreSQL can use to enforce or access data.

3.9 Managed PostgreSQL Still Leaves Schema Design to You

Supabase operates PostgreSQL infrastructure and adds services, but it cannot know what `priority` values your system should accept or whether deleting a user should remove tickets. The platform may expose a table editor, yet the SQL definition is the durable artifact.

Use the dashboard to inspect and learn. Preserve schema changes in ordered SQL files so another environment can be rebuilt and reviewed.

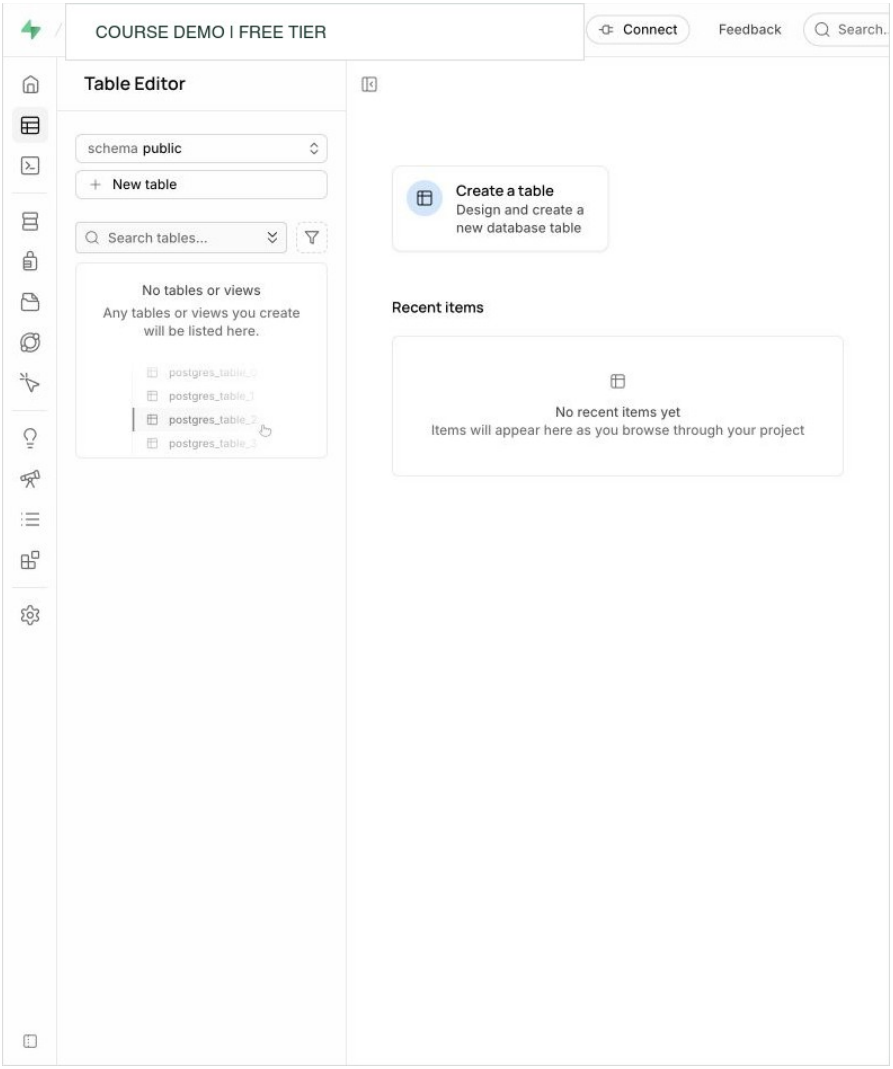

Figure 3.1: Supabase Table Editor with no tables displayed in the selected public namespace. Custom schemas may contain other tables. Account identifiers are redacted. Interface captured August 25, 2026.

The graphical editor helps reveal tables and columns, while the durable definition states names, types, defaults, keys, checks, and referential actions in reviewable SQL. An ordered setup script also makes the schema reproducible in another environment.

3.10 Worked Example: Audit the Metro Support Baseline

The setup script accepts any text for `status`. That creates plausible but inconsistent states:

The next steps are a worked migration on a fresh practice baseline, before the Week 3 status constraint has been added. Do not run the entire chapter as one script; some listings are definitions or deliberate failures. In particular, keep expected-error statements separate from a normal successful run.

Listing 27. SQL

```
UPDATE metro_support.tickets
SET status = 'IN PROGRESS'
WHERE ticket_id = 1002;
```

The update succeeds even though existing data uses `in_progress`. Reports that filter the expected value may silently miss the row.

3.10.1 Step 1: Inspect Existing Values

Listing 28. SQL

```
SELECT status, count(*)
FROM metro_support.tickets
GROUP BY status
ORDER BY status;
```

3.10.2 Step 2: Normalize Before Constraining

For this known spelling error, map the identified invalid value explicitly. Do not apply a broad text transformation and assume all resulting words represent approved states. An unknown value should be investigated rather than silently reinterpreted.

Listing 29. SQL

```
UPDATE metro_support.tickets
SET status = 'in_progress'
WHERE ticket_id = 1002 AND status = 'IN PROGRESS'
RETURNING ticket_id, status;
```

3.10.3 Step 3: Add the Invariant

Listing 30. SQL

```
ALTER TABLE metro_support.tickets
ADD CONSTRAINT tickets_status_allowed
CHECK (status IN ('new', 'open', 'in_progress', 'resolved', 'closed'));
```

3.10.4 Step 4: Verify With an Expected Failure

Listing 31. SQL

```
INSERT INTO metro_support.tickets (
    ticket_id, requester_id, category, priority, status, subject, opened_at
) VALUES (
    1099, 101, 'parks', 'low', 'almost_done', 'Constraint test', now()
);
```

The expected constraint error confirms that the database rejects the bad state. Run the test inside a transaction and roll it back if any part succeeds.

3.11 Inspect Metadata Instead of Guessing

The catalog is data about the database. `information_schema` offers portable views; PostgreSQL's `pg_catalog` exposes deeper implementation details.

Listing 32. SQL

```sql
SELECT
    column_name,
    data_type,
    is_nullable,
    column_default
FROM information_schema.columns
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets'
ORDER BY ordinal_position;
```

Listing 33. SQL

```sql
SELECT indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'metro_support'
  AND tablename = 'tickets';
```

Metadata queries are more trustworthy than remembering what a dashboard displayed or assuming a script ran.

3.12 Common Misconceptions

3.12.1 "The application validates it, so the database does not need to"

Imports, scripts, future services, bugs, and administrator actions can bypass one application. Enforced database constraints apply regardless of which ordinary client issues a write. An administrator who can alter or remove the rules is a different trust boundary; constraint design does not replace access control.

3.12.2 "More constraints are always better"

An incorrect rule can reject valid business states and make change difficult. Add constraints for stable invariants, name them, test existing data, and plan change.

3.12.3 "Every foreign key should cascade"

Cascade is convenient but may erase more than intended. Choose based on lifecycle and audit requirements.

3.13 Practice

The assigned Week 3 labs use this chapter to inspect the actual baseline and then add priority/status rules with valid and invalid tests. Before Day 1, read through **Keys Identify and Connect Facts**, including the normalization review. Before Day 2, read **Constraints Reject Invalid States** and the worked status repair. The broader proposals below are optional study practice.

Audit `metro_support.tickets` and propose:

1. one value-set constraint;
2. one nullability decision;
3. one referential-action decision; and

4. one candidate index tied to a stated query.

For each proposal, name the bad state or workload it addresses and one tradeoff.

3.14 Retrieval and Transfer

1. What is the difference between a PostgreSQL schema and a table definition?
2. Why does a foreign key permit null unless the column is also `NOT NULL`?
3. How is a `UNIQUE` constraint conceptually different from an index?
4. What test confirms that a new constraint is working?
5. Which Metro Support rule is too complex for a simple row-level `CHECK`, and why?

3.15 Further Reading

- Andy Pavlo, Carnegie Mellon University, **Functional Dependencies** and **Normal Forms** (2017 lecture notes): https://15445.courses.cs.cmu.edu/fall2017/notes/04-notes-functionaldependencies.pdf and https://15445.courses.cs.cmu.edu/fall2017/notes/05-notes-normalforms.pdf. These free notes extend the dependency and decomposition discussion; they are external readings rather than course-authored material.
- PostgreSQL data definition
- PostgreSQL constraints
- PostgreSQL schemas
- PostgreSQL indexes
- Supabase database overview

Part II: Operating PostgreSQL Safely

Once data meaning is explicit, administration becomes controlled change under concurrency, access rules, performance constraints, and failure. Chapters 4-8 move from views and migrations through transactions, row-level security, query plans, indexes, backup, and restore.

Each topic uses the same discipline: frame a testable promise, observe a baseline, make one deliberate change, verify the result and side effects, and record what the test does not cover. Supabase appears as managed PostgreSQL, not as a new SQL dialect, so provider responsibility remains separate from customer-owned schema, permissions, queries, secrets, and recovery.

4 Safe Changes Are Planned and Verified

4.1 Operating Question

How can a team change a database while limiting broken queries, bad data, long locks, and changes that cannot be undone?

An application rarely stops existing when a new database requirement arrives. Old clients continue to send requests, reports continue to run, and yesterday's records must still mean what they meant yesterday. A migration is therefore a transition between usable states, not merely the final `ALTER TABLE` statement. We will use views and a newly required field to examine that transition, then connect generated identifiers to what a transaction can and cannot roll back.

4.2 Learning Outcomes

After this module, you can:

- use views to create a stable and limited query interface;
- explain identity columns and sequences without confusing them with row counts;
- inspect columns, constraints, views, and dependencies through metadata;
- plan a small migration using precheck, change, verification, and rollback; and
- apply an expand-migrate-contract pattern to a breaking change.

4.3 A Database Change Has More Than One Audience

Adding a column may affect:

- existing rows that do not have a value;
- application code that selects or inserts columns;
- reports and views that depend on names or types;
- locks and query latency while the change runs;
- backup and recovery procedures;
- permissions and row-level policies; and
- people trying to understand the current schema.

The SQL statement is only one part of a migration. A professional change includes intent, preconditions, execution, verification, and a response if the result is wrong.

4.4 Views Create a Query Interface

A view stores a query definition, not a separate copy of its result in the common case.

This reading uses `chapter4_active_queue`. The classroom lab uses the separate name `active_ticket_queue`, so you can run both examples without replacing one with a different column layout. Chapter 6 reuses this reading's view.

Listing 34. SQL

```sql
CREATE OR REPLACE VIEW metro_support.chapter4_active_queue AS
SELECT
    t.ticket_id,
    t.category,
    t.priority,
    t.status,
    t.subject,
    t.opened_at,
    u.display_name AS assignee_name
FROM metro_support.tickets AS t
LEFT JOIN metro_support.users AS u
    ON u.user_id = t.assignee_id
WHERE t.status IN ('new', 'open', 'in_progress');
```

The view can:

- hide an underlying join;
- expose only approved columns;
- give a stable name to a common query; and
- separate a consumer's interface from some storage details.

A view is not automatically faster. PostgreSQL usually expands its query into the outer query and plans the whole statement. A materialized view stores results and requires refresh, which creates a freshness tradeoff.

4.4.1 Follow a Read Through the View

On the full baseline, the active queue returns seven tickets, including the two that do not yet have an assigned agent. The left join preserves those requests; the `assignee_name` expression becomes null for them. The view does not turn that null into a nonexistent user. An application can display "Unassigned" while retaining the ticket's identity.

If we update a ticket's status to `resolved`, a subsequent ordinary view query uses the current visible database state and stops including that ticket. There is no separate queue table for us to synchronize. A materialized view is different: it holds previously computed results, so an application needs a refresh policy and an acceptable staleness limit. Both mechanisms can be useful, but they have different meanings when someone asks whether the screen is current.

A view also does not automatically isolate secrets. Omitting email from a reporting view helps define a narrower interface, but a role that can select the underlying `users` table can still retrieve email directly. Chapter 6 tests the combination of the view and the grants as the intended actor.

4.4.2 Avoid `SELECT *` in Durable Views

Explicit columns make the contract visible. PostgreSQL expands `SELECT *` when it defines a view: adding a base-table column later does **not** automatically add that column to the existing view. However, recreating or replacing a view from a `SELECT *` definition after the table changes can produce a different interface. Naming the intended columns makes that review deliberate, especially when the new column contains private information.

In PostgreSQL, `CREATE OR REPLACE VIEW` must preserve existing output column names, order, and types, although it can append columns. Inserting a new column in the middle of an established definition is not equivalent to appending it. A consumer that depends on a name or position may break even if the new query returns sensible values. For a genuinely incompatible interface, a separately named version can let old and new consumers coexist while they migrate.

Ordering is another part of the contract to state explicitly. A view's rows are not inherently in a guaranteed presentation order. A consumer that needs the newest requests should request `ORDER BY opened_at DESC, ticket_id` in its own query rather than infer an ordering from earlier observations.

4.5 Identity Columns and Sequences Generate Values

An identity column asks PostgreSQL to generate a value, usually from a sequence:

Listing 35. SQL

```
CREATE TABLE metro_support.chapter4_change_notes (
    note_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    change_name text NOT NULL,
    recorded_at timestamptz NOT NULL DEFAULT now()
);
```

Generated identifiers are not row counts. Gaps are normal because sequence values may be consumed by rolled-back transactions, failed inserts, caching, or manual allocation. Do not promise consecutive numbers or infer how many rows exist from the largest identifier.

`chapter4_change_notes` is a reading-only practice table; the lab creates its own `change_notes` table. Create this table once in your disposable course database. If it already exists, inspect its rows rather than expecting a new sequence to start at 1.

`GENERATED ALWAYS` resists explicit values unless overridden. `GENERATED BY DEFAULT` permits an explicit value. Choose according to import and ownership needs.

4.5.1 Why a Rollback Leaves a Gap

Suppose the table above has just been created and its identity sequence has not been used. Run this sequence against that practice table:

Listing 36. SQL

```
BEGIN;
INSERT INTO metro_support.chapter4_change_notes (change_name)
VALUES ('Initial view') RETURNING note_id;
COMMIT;

BEGIN;
INSERT INTO metro_support.chapter4_change_notes (change_name)
VALUES ('Rehearsal that will not be retained') RETURNING note_id;
ROLLBACK;

BEGIN;
INSERT INTO metro_support.chapter4_change_notes (change_name)
VALUES ('Approved revision') RETURNING note_id;
COMMIT;

SELECT note_id, change_name
FROM metro_support.chapter4_change_notes ORDER BY note_id;
```

The inserts return 1, 2, and 3, but the final table contains rows 1 and 3. The second row was rolled back. Its sequence allocation was not. With multiple sessions, reclaiming a number every time one transaction aborts would complicate independent allocation. A sequence provides generated values without promising an uninterrupted count of committed rows. The PostgreSQL sequence documentation describes this non-rollback behavior.

The explicit commit boundaries matter when an editor sends this entire listing as one batch. The first row must be committed before the rehearsal begins; otherwise a client-side batch can place earlier statements in the transaction that is later rolled back. Transaction boundaries are part of the example, not just formatting.

On a table already used in a previous run, the exact numbers will be higher. The interpretation remains the same: use `count(*)` to count rows, and use keys to identify them. Requirements for consecutive invoice or receipt numbers need a separately designed business process; an identity column is not that process. Likewise, explicitly importing a large ID does not automatically advance the underlying sequence to that value. Imports need a deliberate sequence check.

4.6 Introspection Reveals the Actual State

Before changing an object, query the system that will be changed.

4.6.1 Find Columns and Defaults

Listing 37. SQL

```sql
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets'
ORDER BY ordinal_position;
```

4.6.2 Find Constraints

Listing 38. SQL

```sql
SELECT constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets'
ORDER BY constraint_type, constraint_name;
```

4.6.3 Find Views That Mention a Table

Listing 39. SQL

```sql
SELECT table_schema, table_name, view_definition
FROM information_schema.views
WHERE view_definition ILIKE '%metro_support.tickets%';
```

Text search is a useful first pass, not a complete dependency model. PostgreSQL catalogs and tools can expose more precise dependencies. The key habit is to inspect rather than assume.

4.7 What a Migration Must Establish

For a small migration, be able to explain:

1. **Intent:** what user or operating need justifies the change?
2. **Precheck:** what must be true before execution?

3. **Change:** what ordered SQL creates the new state?
4. **Verification:** what proves structure and data are correct?
5. **Rollback or forward fix:** how will the team respond if verification fails?

Not every production migration can be rolled back instantly. A forward fix may be safer after data has been transformed. State the boundary honestly.

4.8 Worked Example: Add a Public Status Label Safely

Metro Support wants user-facing labels such as “In progress” while preserving the machine value `in_progress`.

4.8.1 Risky Approach

Rename or replace the stored values immediately. Existing queries, constraints, and integrations may expect the old values.

4.8.2 Expand

Add a view with a derived label while preserving the source column:

Listing 40. SQL

```sql
CREATE OR REPLACE VIEW metro_support.public_ticket_status AS
SELECT
    ticket_id,
    subject,
    status AS status_code,
    CASE status
        WHEN 'new' THEN 'New'
        WHEN 'open' THEN 'Open'
        WHEN 'in_progress' THEN 'In progress'
        WHEN 'resolved' THEN 'Resolved'
        WHEN 'closed' THEN 'Closed'
        ELSE 'Unknown'
    END AS status_label,
    opened_at,
    closed_at
FROM metro_support.tickets;
```

4.8.3 Verify

Listing 41. SQL

```sql
SELECT status_code, status_label, count(*)
FROM metro_support.public_ticket_status
GROUP BY status_code, status_label
ORDER BY status_code;
```

Check that every known code has one intended label and that the total count equals the base table count.

4.8.4 Migrate Consumers

Update a report or application to read the view. Observe before removing or renaming any old interface.

4.8.5 Contract Later

Only after all consumers are confirmed should an obsolete interface be removed. In this case, keeping both code and label is useful, so contraction may not be needed.

4.9 Transactional DDL Helps, but Locks Still Matter

Many PostgreSQL data-definition statements can run inside a transaction:

The next example rehearses the required-field change from the Week 4 lab. Begin with the practice baseline before `source_channel` exists. Historical tickets have no structured channel field. Some event notes mention mobile submission, so it would be false to label every old ticket `web`. We use `unknown` to mean that the structured value was not captured. More detailed backfilling would require a reviewed, record-specific source.

Listing 42. SQL

```sql
BEGIN;
SET LOCAL lock_timeout = '2s';
SET LOCAL statement_timeout = '10s';

ALTER TABLE metro_support.tickets
ADD COLUMN source_channel text;

UPDATE metro_support.tickets
SET source_channel = 'unknown'
WHERE source_channel IS NULL;

ALTER TABLE metro_support.tickets
ADD CONSTRAINT tickets_source_channel_allowed
CHECK (source_channel IN ('web', 'phone', 'mobile', 'unknown'));

ALTER TABLE metro_support.tickets
ALTER COLUMN source_channel SET DEFAULT 'unknown',
ALTER COLUMN source_channel SET NOT NULL;

-- Inspect before choosing COMMIT or ROLLBACK.
SELECT source_channel, count(*)
FROM metro_support.tickets
GROUP BY source_channel;

ROLLBACK;
```

The grouped result inside this rehearsal is `unknown`, 12. After rollback, the new column and its constraint no longer exist. To retain the migration, repeat the verified block and deliberately choose `COMMIT` instead. Do not run the creation block repeatedly against an already migrated table.

The default has a particular purpose: an older client that inserts an explicit list of the old columns can still create a row. Its omitted channel receives `unknown`. An explicit null is different from an omitted field and fails the new not-null rule. A new client should provide the actual channel. After all writers are upgraded, we can reconsider whether the default still serves a useful purpose. Adding a default is an application-compatibility decision, not just a way to make an error disappear.

Rollback protects atomicity, but an `ALTER TABLE` may still acquire a strong lock and block other work while the transaction remains open. Production-safe change planning considers table size, lock duration, statement timeout, maintenance windows, and compatibility with live clients.

4.9.1 A Safer Pattern for Large Tables

For a required new value:

1. add a nullable column;
2. deploy writers that populate it;
3. backfill old rows in controlled batches;
4. verify no nulls remain;
5. add or validate a constraint; and
6. remove old behavior only after consumers migrate.

The exact feature support and lock behavior depend on the PostgreSQL version. Consult current official documentation for a production plan.

4.9.2 Before Commit and After Commit Are Different Repair Boundaries

During our small rehearsal, rollback restores the prior database state. After a deployment commits and new mobile requests arrive, dropping `source_channel` would erase newly collected facts. That may be technically possible but is not an adequate recovery plan. A forward repair could correct the view definition or writer behavior while retaining channel data.

The same distinction applies to a destructive transformation. If a script replaces detailed values with a coarse category, reversing its SQL does not reconstruct the discarded detail. Preserve a verified source or choose a non-destructive transition when that information matters. A rollback plan must describe the state and data it recovers, rather than merely name an opposite command.

The timeouts in the classroom block are guardrails for this small experiment. `lock_timeout` limits waiting to acquire a lock; `statement_timeout` limits a statement's elapsed execution. A timeout can leave the current transaction aborted, requiring rollback. Their appropriate production values depend on the workload and operational policy. They do not make a long migration intrinsically safe or eliminate the need to understand its locks.

4.10 Verification Should Cover Structure and Meaning

Run the following checks while the migrated column exists, either inside the rehearsal before its rollback or after an intentionally committed migration. After rollback, the column's absence is the expected result.

Structural check:

Listing 43. SQL

```sql
SELECT column_name, is_nullable
FROM information_schema.columns
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets'
  AND column_name = 'source_channel';
```

Data check:

Listing 44. SQL

```sql
SELECT count(*) AS missing_source_count
FROM metro_support.tickets
WHERE source_channel IS NULL;
```

Behavior check:

Listing 45. SQL

```sql
BEGIN;
INSERT INTO metro_support.tickets (
    ticket_id, requester_id, category, priority, status, subject, opened_at
) VALUES (
    1098, 101, 'parks', 'low', 'new', 'Old-client compatibility test', now()
)
RETURNING ticket_id, source_channel;
ROLLBACK;
```

With our compatibility default, this insert succeeds and returns `unknown` for the channel. The rollback removes the test ticket. A second test that explicitly supplies a null channel should fail the not-null rule. If we later remove the default, this omitted-field insertion will fail too, so that change requires knowing that old writers have been upgraded.

4.11 Common Misconceptions

4.11.1 "A successful migration statement proves the application is safe"

The structure may have changed while existing queries, permissions, or data are wrong. Verify at multiple layers.

4.11.2 "Sequences should never have gaps"

Sequences allocate values efficiently, not gapless business numbering. A primary key or unique constraint enforces uniqueness in the stored table, including against explicit values or later sequence resets.

4.11.3 "A view is a backup"

A normal view stores a query definition. It depends on underlying objects and does not preserve deleted data.

4.12 Practice

For the assigned Week 4 labs, read **Views Create a Query Interface** through **Introspection Reveals the Actual State** before Day 1. Before Day 2, read the migration sections, especially the historical-channel and old-client examples. The resolution-summary problem below is optional transfer practice.

Plan a change that adds `resolution_summary` for resolved or closed tickets. Write:

1. the intent;
2. one data precheck;
3. an expand step;
4. two verification queries; and
5. a rollback or forward-fix decision.

Do not make the field required in the same step that creates it. Explain why.

4.13 Retrieval and Transfer

1. What problem does a view solve that a base table does not?
2. Why can a generated identity value contain gaps?
3. Name the five parts of a small change record.
4. Why does transactional DDL not eliminate operational risk?
5. A new required column will be added to ten million rows. Which expand-migrate- contract steps reduce risk?

4.14 Further Reading

- PostgreSQL views
- PostgreSQL identity columns
- PostgreSQL information schema
- PostgreSQL `ALTER TABLE`
- Supabase database migrations

5 Transactions Coordinate Competing Work

5.1 Operating Question

When two sessions read and change the same data, how can we predict what each session sees, identify who is waiting, and resolve the situation without guessing?

5.2 Learning Outcomes

After this module, you can:

- define a transaction boundary and demonstrate `COMMIT` and `ROLLBACK`;
- connect atomicity, consistency, isolation, and durability to observable behavior;
- explain snapshots and row versions at a beginner level;
- distinguish normal waiting, blocking, and deadlock;
- use PostgreSQL activity and blocking state to identify session relationships; and
- resolve a controlled blocking incident safely and verify the result.

5.3 A Transaction Is a Unit of Decision

A transaction groups database statements into one unit that either commits or rolls back.

Consider a repair desk assigning an unassigned request to Noah. The application needs to change the ticket's current assignee and add an event saying who made that change. If the ticket update succeeds but the history insert fails, the current state and the history disagree. Each statement can be valid SQL, and each table can satisfy its own constraints, while the application operation is still incomplete. A transaction lets the application define that larger unit.

Start with the smaller boundary below. It uses the supplied Metro Support data and ends with rollback so reading the example does not permanently change the fixture. Run the block as a whole in your own practice database.

Listing 46. SQL

```sql
BEGIN;

UPDATE metro_support.tickets
SET assignee_id = 202,
    status = 'in_progress'
WHERE ticket_id = 1004
RETURNING ticket_id, assignee_id, status;

ROLLBACK;
```

Many clients use **autocommit**: each statement becomes its own transaction unless you explicitly begin one. An explicit transaction creates a review point, but it also keeps locks and snapshots alive until the transaction ends. "I forgot to commit" is operationally meaningful.

The `RETURNING` row is visible to the transaction that made the change. It is not a commit receipt. After this block, a fresh query should still show ticket 1004 as unassigned and `new`. Replacing the last statement with `COMMIT` keeps the change, but only do that when the exercise asks for it. The Day 1 lab works in separate disposable copies so its committed example does not alter this source.

5.3.1 Keeping State and History Together

This second example adds the history event. It assumes the original fixture: ticket 1004 is `new` and event 5998 does not exist.

Listing 47. SQL

```sql
BEGIN;

UPDATE metro_support.tickets
SET assignee_id = 202, status = 'in_progress'
WHERE ticket_id = 1004 AND status = 'new'
RETURNING ticket_id, assignee_id, status;

INSERT INTO metro_support.ticket_events
    (event_id, ticket_id, actor_id, event_type, old_status,
     new_status, note, event_at)
VALUES
    (5998, 1004, 202, 'status_changed', 'new',
     'in_progress', 'Assigned to Noah for follow-up', now());

SELECT event_id, ticket_id, new_status
FROM metro_support.ticket_events WHERE event_id = 5998;
ROLLBACK;
```

Inside the transaction, both changes can be inspected. Outside it after rollback, neither remains. If the insert violates a constraint, PostgreSQL reports an error and the transaction enters a failed state. End it with `ROLLBACK`; do not keep issuing ordinary statements and assume that the earlier update committed.

There is an important boundary to this protection. An `UPDATE` matching zero rows is **not** an SQL error. A real application must check that the intended ticket was changed before accepting the history event and committing. The `status = 'new'` condition protects the proposed transition, but it does not automatically tell later application code to stop when the condition matches nothing. The database cannot infer the meaning of "assign this previously unassigned request" from an arbitrary list of successful statements.

5.3.2 What Rollback Does Not Undo

Rollback covers transactional database changes. It does not retract an email already sent, undo an external API call, or reclaim every sequence value. Chapter 4's identity example showed a sequence gap after rollback. Chapter 14 returns to external messages: an application must coordinate them with database commits rather than assume that a transaction spans every service it calls.

A rollback should be verified, not merely assumed. Figure 5.1 shows a reversible browser exercise that creates a temporary table, inserts one row, rolls the transaction back, and then asks PostgreSQL whether the temporary relation still exists. The returned `true` means `to_regclass(...) IS NULL`: the relation is no longer present.

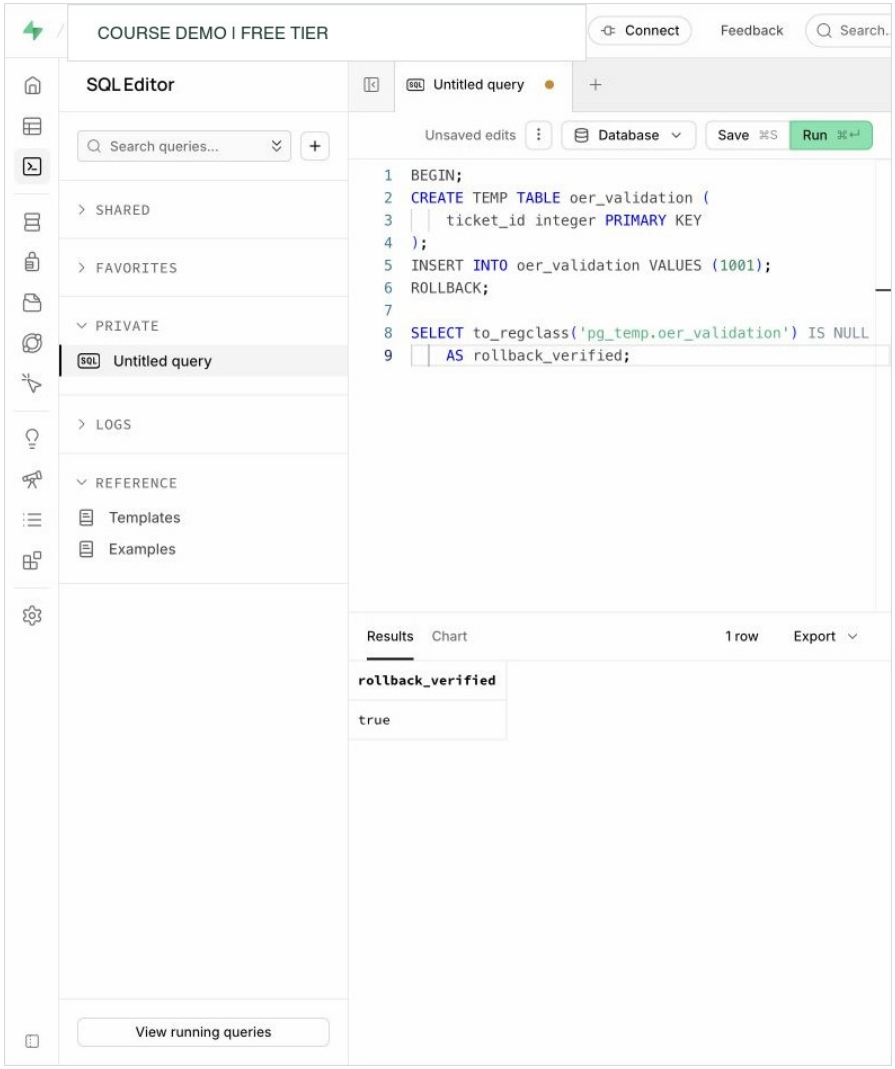

Figure 5.1: A Supabase SQL Editor transaction test verifies that rollback removed the temporary relation. The query changes no persistent course data. Account identifiers are redacted. Interface captured August 25, 2026.

This is stronger than a green "success" message because the final `SELECT` checks the postcondition independently. Supabase may still display a heuristic RLS warning for a `CREATE TABLE` statement; in this controlled example the table is temporary and the transaction is rolled back. A persistent application table would require an explicit access-policy decision.

5.4 ACID Describes Guarantees, Not a Product Label

5.4.1 Atomicity

All changes in a transaction commit together or none remain. If assigning a ticket and recording its event must be one operation, put them in one transaction.

5.4.2 Consistency

A transaction moves the database between states that satisfy enforced rules. Consistency is not magic correctness: the database can enforce only the constraints and transaction logic it has been given.

In this use of the word, consistency means maintaining application invariants, such as a valid ticket status and a matching history event. Later chapters use "consistency" when comparing observations across replicas. The same word refers to different questions; a valid state is not automatically an immediately visible state on every machine.

5.4.3 Isolation

Concurrent transactions behave according to an isolation model. Isolation does not always mean that every transaction runs as though completely alone. Different levels permit different observations and may require retries.

5.4.4 Durability

After a successful commit, the DBMS promises the change survives failures within its documented durability model. Durability is not the same as indefinite retention or protection from an authorized later deletion. Backups and recovery still matter.

5.5 Concurrency Creates Useful Work and New Questions

Without concurrency, a database would waste time while one client thinks, communicates, or waits for input. With concurrency, two sessions may:

- read the same row;
- update different rows;
- compete to update the same row;
- make decisions from different snapshots; or
- acquire resources in opposite orders.

Common anomaly names include dirty read, nonrepeatable read, phantom, lost update, and serialization anomaly. PostgreSQL's implementation and isolation levels determine which can occur. The practical questions are:

1. what did each transaction read?
2. what did each transaction change?
3. which transaction is waiting, and on whom?
4. which outcome can the application accept?

The names become useful when attached to a situation. A **dirty read** would show another transaction's uncommitted assignment. A **nonrepeatable read** shows an old assignment on one query and a newly committed assignment on a later query. A **phantom** concerns the membership of a result: the next active-ticket query includes a newly committed request. These are not all the same error. For a live queue, seeing a newly committed ticket may be exactly what the application wants. For two parts of one financial report, changing the visible data between queries may be unacceptable.

5.6 MVCC Lets Readers and Writers Coexist

PostgreSQL uses multi-version concurrency control, or MVCC. An update creates a new row version rather than overwriting the only copy in place. A transaction reads versions visible to its snapshot.

This design often allows readers and writers to proceed without blocking one another. It does not mean there are no locks. Writers competing for the same row can wait, schema changes can take strong locks, and long transactions can prevent old row versions from being cleaned up.

`VACUUM` helps reclaim or mark reusable space from obsolete versions and supports transaction-ID safety. Routine vacuuming is normal database maintenance; it does not by itself mean that PostgreSQL is broken.

Think of a row version as a value with visibility conditions, not a second ticket the user should count. While Noah's assignment is uncommitted, an ordinary reader can still see the previously committed ticket. After commit, a later snapshot can see the new version. Old snapshots may still need the old version, which is one reason a long-running transaction can interfere with cleanup even if it is not actively changing rows. MVCC is not a built-in historical reporting API: obsolete versions are eventually reclaimed rather than kept as a permanent event history.

5.7 Isolation Levels Change Visibility and Retry Behavior

PostgreSQL supports:

- **Read Committed**, the default: each statement receives a snapshot at statement start. Two selects in one transaction may see different committed data.
- **Repeatable Read:** the transaction keeps a stable snapshot for ordinary reads; conflicting patterns may produce an error rather than an inconsistent result.
- **Serializable:** PostgreSQL attempts behavior equivalent to some serial order; applications must be prepared to retry serialization failures.

PostgreSQL treats Read Uncommitted as Read Committed. Do not infer behavior only from a generic isolation-level chart; consult the DBMS documentation.

Stronger isolation is not simply "better." It can increase coordination, abort/retry behavior, and application complexity. Choose based on the invariant.

5.7.1 Read the Same Row Twice

Here is a reasoning example, not an instruction to leave browser sessions open. Both sessions start from ticket 1004 with priority `low`.

Order	Session A	Session B
1	begins a transaction and reads `low`	not yet changing the row
2	remains open	changes priority to `high` and commits
3	reads the same ticket again	has finished
4	ends its transaction	has finished

With PostgreSQL Read Committed, A's second read can show `high`: the second statement gets a new snapshot. With Repeatable Read, A's ordinary second read still shows `low`: its transaction uses the snapshot established by its first non-control statement. B's commit was not lost; it is simply outside A's snapshot. A new transaction can see it. If A now tries to update a row changed since that Repeatable Read snapshot, a serialization failure may require A to retry. PostgreSQL isolation behavior

5.7.2 A Stable Snapshot Is Not Every Business Rule

Suppose Priya and Noah are both on call, and the rule is "at least one agent must remain on call." Each transaction reads that the other agent is available and then changes only its own agent to off call. Under snapshot isolation, the writes can target different rows and both commit, leaving nobody on call. There was no dirty read, and each transaction's view was stable, but the shared rule failed. This pattern is called **write skew**.

The point is not to memorize another name. Identify whether the rule concerns one row or a relationship among several decisions. A design can coordinate on a shared locked record or use serializable transactions with a retry path. The right choice depends on the actual invariant. Merely adding `BEGIN` and `COMMIT` around a read-then-write program does not establish serial execution.

5.8 Waiting Is a Relationship Between Sessions

If Session A updates ticket 1004 and remains open, Session B's attempt to update the same row waits. Session B is blocked; Session A is the blocker.

Waiting can be correct: competing row writes must coordinate. It does not by itself prevent a stale application value from overwriting a newer value after the wait ends. The problem may be an unexpectedly long wait, an unknown transaction, or a write that no longer satisfies the application's condition.

The two-session SQL below explains the mechanism. Use it only with two persistent connections to a disposable practice database and release Session A promptly. Separate web-editor tabs are not guaranteed to represent persistent independent database sessions. For the assigned exercise, Notebook 02 creates the wait, captures it, and releases it within one cell. Reading its output does not leave another query blocked.

5.8.1 Session A

Listing 48. SQL

```sql
BEGIN;

UPDATE metro_support.tickets
SET priority = 'high'
WHERE ticket_id = 1004;

-- Leave this transaction open only during the controlled lab.
```

5.8.2 Session B

Listing 49. SQL

```sql
BEGIN;
SET LOCAL statement_timeout = '15s';
UPDATE metro_support.tickets
SET status = 'in_progress'
WHERE ticket_id = 1004;
COMMIT;
```

Session B waits because the target row is already being changed by the uncommitted transaction in Session A.

If Session B times out, issue `ROLLBACK` in B and release A before retrying the exercise. A timeout is a bound on waiting, not evidence that the blocking transaction was automatically removed.

5.9 Diagnose From a Third Session

`pg_stat_activity` exposes server processes and current activity. Access may be limited by role and managed-service policy.

Listing 50. SQL

```sql
SELECT
    pid,
    usename,
    state,
    wait_event_type,
    wait_event,
    xact_start,
    query_start,
    left(query, 100) AS query_sample
FROM pg_stat_activity
WHERE datname = current_database()
ORDER BY xact_start NULLS LAST, query_start;
```

Use `pg_blocking_pids` to ask PostgreSQL for the relationship:

Listing 51. SQL

```sql
SELECT
    blocked.pid AS blocked_pid,
    blocked.wait_event_type,
    blocked.wait_event,
    pg_blocking_pids(blocked.pid) AS blocking_pids,
    left(blocked.query, 100) AS blocked_query
FROM pg_stat_activity AS blocked
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0;
```

The process identifier is a diagnostic clue, not permission to terminate a session. First identify the user, transaction age, query, application, impact, and owner.

5.10 Worked Example: Resolve the Controlled Block

1. Confirm that Session B is waiting and record its PID.
2. Record the blocking PID returned by `pg_blocking_pids`.
3. Return to Session A and decide whether its change is valid.
4. Use `ROLLBACK` in the controlled lab to release the row lock without keeping the priority change.
5. Observe Session B finish.
6. Query ticket 1004 from a fresh session and record the final priority and status.

The final state proves which change remained. The activity view proves the blocking relationship. Neither alone tells the complete incident story.

5.11 Deadlock Is Different From One-Way Blocking

A deadlock forms a cycle: Session A holds something Session B needs while Session B holds something Session A needs. Neither can proceed. PostgreSQL detects the cycle and aborts one transaction so the other can continue.

Applications should handle the error and retry the entire transaction when safe. Design can reduce deadlocks by touching resources in a consistent order and keeping transactions short.

Do not create uncontrolled deadlocks in a shared environment. A controlled two-row exercise belongs in a dedicated practice schema with explicit cleanup.

5.12 Safe Incident Communication

A useful update separates observation from inference:

> Ticket updates are waiting. At 14:12 UTC, PID 8124 was waiting on a transaction lock and `pg_blocking_pids` identified PID 8071. The blocking transaction began at 14:06 UTC from the course SQL editor. We rolled back the controlled Session A change, Session B completed, and a fresh query confirmed ticket 1004 is `in_progress` with its original priority. We will add a transaction timeout and shorten the workflow before repeating the test.

Avoid "the database froze" when the system views show one lock relationship.

5.13 Common Misconceptions

5.13.1 "Readers never block and writers never wait under MVCC"

MVCC reduces many read/write conflicts. Row writes, explicit locks, and schema changes can still block.

5.13.2 "The oldest query is always the blocker"

Age is a clue. Use blocking relationships and lock state, not age alone.

5.13.3 "Killing the blocked session fixes the cause"

The blocked session is the waiter. Terminating it may remove the symptom while the blocking transaction remains open.

5.13.4 "Serializable means no errors"

Serializable execution can abort a transaction to preserve the guarantee. Correct applications expect and safely retry those failures.

5.14 Study and Practice

The Week 5 guide identifies assigned reading and links the required individual labs. The following timeline is optional self-study, not an additional report.

Draw a timeline for Session A and Session B in the worked example. Mark:

- transaction start;
- snapshot or statement reads;
- row update;
- wait start;
- rollback; and
- final verification.

Then write one query you would run in the third session and one conclusion it can support.

5.15 Retrieval and Transfer

1. Why can an explicit transaction be both safer and riskier than autocommit?
2. What does MVCC mean for row updates?
3. How does one-way blocking differ from deadlock?
4. Which function directly identifies PostgreSQL blocking PIDs?
5. Why must a serializable application be able to retry?
6. What final check proves which ticket state remained after the incident?

5.16 Further Reading

- PostgreSQL transactions
- PostgreSQL transaction isolation
- PostgreSQL explicit locking
- PostgreSQL monitoring statistics
- PostgreSQL routine vacuuming

6 Access Should Be Useful and Limited

6.1 Operating Question

How can we prove that the right actor can perform a required action while the wrong actor is denied, without exposing credentials during the test?

6.2 Learning Outcomes

After this module, you can:

- distinguish identity, authentication, authorization, and auditing;
- represent access as actor-action-resource conditions;
- create PostgreSQL roles and apply grants using least privilege;
- explain the relationship among Supabase Auth, PostgreSQL roles, and row-level security;
- write and test a beginner RLS policy; and
- handle connection strings and service credentials safely.

6.3 Security Questions Need Separate Terms

Imagine two residents using the same repair application. Both have valid accounts; both need to read their requests. If one resident changes a ticket number in a browser URL, the server must still decide whether that person may read the requested row. Hiding a button or using a long identifier does not make the row private. The access decision must hold when the caller sends a different request.

- **Identity:** who or what claims to be acting?
- **Authentication:** how is that claim verified?
- **Authorization:** what may the authenticated actor do?
- **Auditing:** which logs record relevant actions and decisions?

A valid password proves authentication only within its mechanism. It does not mean the account should read every row. A failed query may reflect authentication, authorization, network access, an object name, or SQL syntax. Classify the failure before changing permissions.

These stages explain several different symptoms. “Network unreachable” occurs before the server can decide table permissions. A bad password prevents the intended login. “Permission denied for table” means a reachable database rejected an operation. A query returning no rows may be valid because no row satisfies an access policy. Granting every privilege is not an appropriate response to all four situations.

6.4 Model Access as Actor, Action, Resource, and Condition

Replace “give the app access” with a testable statement:

Actor	Action	Resource	Condition
support agent	`SELECT`, `UPDATE`	assigned tickets	only rows assigned to that agent
resident	`SELECT`	tickets	only tickets requested by that resident
analyst	`SELECT`	reporting view	no private internal notes
migration process	schema changes	application schema	only during controlled deployment

This matrix reveals different layers. A table grant may permit `SELECT` on the table, while RLS limits which rows the statement can return.

A **privilege** authorizes an operation on an object, such as reading a table. A **policy** can add conditions on which rows that operation may affect. For an ordinary SELECT in the simple resident design, think of the result as the rows satisfying both the application’s question and the resident’s ownership rule.

$$R_{\text{visible}} = \sigma_{\text{requested condition} \wedge \text{owner is current resident}}(R).$$

This is a model of the result, not an instruction to trust an ownership condition supplied by the browser. The database policy is applied even when the caller’s query omits that condition. Object privileges must also permit SELECT; a row policy does not grant table access on its own.

6.5 PostgreSQL Roles Represent Users and Groups

PostgreSQL uses roles for both login identities and privilege groups.

Listing 52. SQL

```
CREATE ROLE metro_analyst NOLOGIN;
GRANT metro_analyst TO CURRENT_USER;
```

This creates a permission group and lets your current administrative login assume it during the controlled test. It does not create a new password or an application account. Run it only in your personal practice environment with the permitted role-management privileges. If the role already exists from your earlier run, inspect or clean up that exercise rather than treating an existing shared role as disposable.

A group role with `NOLOGIN` collects privileges. Login roles become members. This separates identity lifecycle from permission definition.

`session_user` identifies the original database login; `current_user` is the effective database role used for checks such as the test below. They can differ after `SET ROLE`. A person in the Supabase dashboard is another identity again. Keeping these names distinct makes an access test interpretable.

6.6 Grant the Minimum Required Privileges

PostgreSQL privileges are object-specific. Accessing a table in a schema can require both schema `USAGE` and table privileges.

Listing 53. SQL

```sql
GRANT USAGE ON SCHEMA metro_support TO metro_analyst;
GRANT SELECT ON metro_support.chapter4_active_queue TO metro_analyst;
```

If the analyst should read only the view, do not also grant broad access to every base table. The view above is introduced in Chapter 4; create it there first, or use the complete limited-view example later in this chapter.

A normal PostgreSQL view generally checks underlying relation access using the view owner's permissions. This can deliberately expose selected columns without granting the caller direct access to the base table. It also means that a view owned by an elevated role is not automatically an RLS-safe resident interface. On PostgreSQL 15 and later, a `security_invoker` view uses the caller's underlying permissions instead. The reporting-view exercise and the resident-policy exercise test different designs, not interchangeable shortcuts.

Revoke privileges that were granted too broadly:

Listing 54. SQL

```sql
REVOKE UPDATE, DELETE
ON metro_support.tickets
FROM metro_analyst;
```

Privileges on future tables are separate from current objects. Default privileges can automate future grants, but they are defined by the object-creating role and must be tested.

Privileges can also come from memberships and `PUBLIC`, the implicit group of all roles. Revoking a direct grant from one role does not erase a privilege it still receives through another path. Therefore, read the effective result of a test instead of treating the presence of a REVOKE statement as a security proof.

6.7 Test Both an Allowed and a Denied Action

An allow-only test is incomplete. In a dedicated practice environment:

Listing 55. SQL

```sql
BEGIN;
SET LOCAL ROLE metro_analyst;

SELECT current_user, ticket_id, status
FROM metro_support.chapter4_active_queue;
ROLLBACK;
```

Then test the denied operation in its own transaction on the same connection:

Listing 56. SQL

```sql
BEGIN;
SET LOCAL ROLE metro_analyst;

-- This should fail if no base-table update privilege was granted.
UPDATE metro_support.tickets
SET priority = 'urgent'
WHERE ticket_id = 1001;

-- Execute this even if the client stops after the expected error.
ROLLBACK;
```

The expected error is an insufficient-privilege error, SQLSTATE `42501`, not a misspelled table or a broken connection. Rollback ends the transaction and its local role setting. Keep actor, query, and transaction on one

connection: separate editor executions may use different pooled sessions. The Week 6 lab includes a small Colab display cell when a web editor hides intermediate results.

6.8 Supabase Adds an Identity Layer to PostgreSQL

Supabase Auth can issue JSON Web Tokens for application users. Requests through Supabase's data APIs are mapped to PostgreSQL roles such as `anon` or `authenticated`, and claims can be read by policy helpers such as `auth.uid()`.

A **token** is a value the client presents with a request. A JSON Web Token, or JWT, carries claims such as a user identifier; the receiving service verifies the token before using those claims. A signed token is not necessarily encrypted. Treat a usable login token as a credential, even when its payload can be decoded. The browser must not be allowed to establish identity merely by writing an arbitrary user ID in the request body.

This creates distinct concepts:

- a Supabase dashboard account administers a project;
- a PostgreSQL role controls database privileges;
- an application user exists in the authentication system;
- a token carries claims about a request; and
- an RLS policy evaluates row access inside PostgreSQL.

Do not treat these identities as interchangeable.

In the Day 2 lab, two PostgreSQL roles make the difference observable without building a login system. The four supplied rows name their owner role; comparing `owner_role = current_user` makes resident 101 see tickets 1 and 2 while resident 102 sees 3 and 4. A normal Supabase application instead serves many people through the shared `authenticated` role and uses verified user claims to distinguish them. It does not need one database role for every resident.

The Supabase SQL editor commonly runs with elevated ownership privileges. Table owners and roles with `BYPASSRLS` can bypass row-level security. A successful query in the editor therefore does not prove what an anonymous or authenticated client can see.

6.9 Row-Level Security Adds a Row Predicate

The following is an **illustrative Supabase adaptation**, not a command to paste into the unchanged Metro Support fixture. That fixture has integer requester IDs, not Supabase Auth UUIDs. A UUID is a 128-bit identifier, commonly displayed as groups of hexadecimal characters; it is not a password.

For this adaptation, assume that a reviewed migration has created and populated `requester_auth_id uuid` with actual application-user identities, the appropriate schema is exposed to the intended API, and SELECT has been explicitly granted to `authenticated`. Adding a nullable column without mapping existing residents would not establish correct ownership.

Listing 57. SQL

```sql
ALTER TABLE metro_support.tickets ENABLE ROW LEVEL SECURITY;

CREATE POLICY residents_read_own_tickets
ON metro_support.tickets
FOR SELECT
TO authenticated
USING (requester_auth_id = (SELECT auth.uid()));
```

`USING` controls which existing rows are visible for the operation. `WITH CHECK` controls which new or changed row states may be created for relevant operations.

With no authenticated user, `auth.uid()` can be NULL. Equality with NULL is not true, so this ownership condition does not select a row. That is the same three-valued logic introduced in Chapter 2, now serving an access boundary.

Listing 58. SQL

```sql
CREATE POLICY residents_insert_own_tickets
ON metro_support.tickets
FOR INSERT
TO authenticated
WITH CHECK (requester_auth_id = (SELECT auth.uid()));
```

Policy design is default-deny after RLS is enabled: if no applicable policy allows the operation, ordinary roles cannot perform it. Verify current Supabase guidance and test through the same role and request path the application uses.

An INSERT also needs an INSERT grant. If it supplies a different user's ID, the `WITH CHECK` condition rejects the new row. A SELECT of another user's existing row ordinarily returns no matching row rather than revealing it and then raising an error. This is why access testing must compare visible identities as well as error messages. Supabase RLS guide

Do not solve a failed ownership test by adding `USING (true)`. That rule allows every row for its covered operation. Multiple permissive policies combine with OR, so one broad policy can undo the restriction expected from another. Owners normally bypass RLS; superusers and roles with `BYPASSRLS` bypass it. RLS is not a defense against an unrestricted database administrator. PostgreSQL row-security rules

6.10 Worked Example: Build an Access Test Matrix

Requirement: an analyst may read ticket identifiers, category, priority, status, and opening time, but not resident email addresses or internal event notes.

6.10.1 Design

Create a limited view:

Listing 59. SQL

```sql
CREATE OR REPLACE VIEW metro_support.analyst_ticket_summary AS
SELECT ticket_id, category, priority, status, opened_at, closed_at
FROM metro_support.tickets;

GRANT USAGE ON SCHEMA metro_support TO metro_analyst;
GRANT SELECT ON metro_support.analyst_ticket_summary TO metro_analyst;
REVOKE ALL ON metro_support.users FROM metro_analyst;
REVOKE ALL ON metro_support.ticket_events FROM metro_analyst;
```

6.10.2 Test

Test	Expected
select from `analyst_ticket_summary`	allowed
select email from `users`	denied
select note from `ticket_events`	denied
update a ticket	denied

6.10.3 Interpret

If the expected allow succeeds and every expected deny fails for permission reasons, the results support the stated boundary. They do not demonstrate protection against a privileged administrator, a leaked credential, a vulnerable function, or an untested API path.

6.11 Secret Handling Is a Database Skill

A connection URI may include host, database, user, and password. Treat the whole string as a secret when it contains credentials.

Safe notebook pattern:

Listing 60. Python

```python
from getpass import getpass

database_url = getpass("Paste the temporary database URL: ")
```

Do not hardcode a secret, print it, save it in notebook output, or place it in a GitHub issue. Use environment variables or platform secret storage in applications.

Supabase distinguishes publishable client keys from secret server keys; older projects may also show legacy `anon` and `service_role` keys. A publishable key identifies an application component, not a particular resident. A user's signed login token supplies the user context. Secret keys and legacy service-role keys provide elevated access and must not be shipped in browser code. Check the key type, not just whether a string is called an "API key." Supabase API-key guide

If a secret reaches Git history, revoke or rotate it first. Removing the visible line does not make the old credential safe.

6.12 Network Controls Complement Authorization

A remote connection crosses several gates. The hostname must resolve; the network must reach the service; TLS must establish the intended protected connection; authentication must succeed; and authorization must permit the operation. Passing one gate does not establish that the remaining gates pass.

Atlas IP access lists, database users, TLS, PostgreSQL connection controls, and platform network restrictions reduce who can reach a service. They do not replace least-privilege database authorization.

For temporary classroom allow-list rules, use the narrowest practical scope and remove them after the activity. An allow-from-anywhere rule may be convenient, but it increases exposure and still requires strong credentials and database permissions.

6.13 Common Misconceptions

6.13.1 "RLS is enabled, so the policy works"

Enablement and a policy definition are only configuration. Test the intended client role, token claims, allowed rows, and denied rows.

6.13.2 "The `authenticated` role identifies one person"

It represents a class of requests. Policies commonly use token claims such as the user ID to distinguish rows.

6.13.3 "A public client key is the same as a service-role key"

They have different powers and intended locations. Treat service credentials as secrets and never expose them in client code.

6.13.4 "More privileges will fix the error"

Broad grants may hide the actual problem and create a security defect. Identify the actor, object, action, and expected policy first.

6.14 Study and Practice

For Day 1, read through "Test Both an Allowed and a Denied Action" and the limited-view worked example. For Day 2, read the Supabase identity, row-security, and credential sections. The weekly labs contain complete practice setup and cleanup. The matrix below is optional self-study, not another required file.

Choose one Metro Support actor. Write an access matrix with two allowed actions and two denied actions. For each action, identify:

- the PostgreSQL object or API resource;
- the grant or policy layer involved;
- the exact test; and
- what the result would and would not prove.

6.15 Retrieval and Transfer

1. How do authentication and authorization differ?
2. Why can a PostgreSQL group role use `NOLOGIN`?
3. Why should an access test include a denied action?
4. What is the difference between `USING` and `WITH CHECK` in RLS?
5. Why can the Supabase SQL editor be a misleading RLS test path?
6. What should happen first after a secret is committed to Git?

6.16 Further Reading

- PostgreSQL roles
- PostgreSQL privileges
- PostgreSQL row security
- Supabase row-level security
- Supabase API security

7 Performance Work Begins With Measurement

7.1 Operating Question

When a query feels slow, what observations can distinguish a scan, a bad estimate, an expensive join, an avoidable sort, blocking, or simply more work than expected?

7.2 Learning Outcomes

After this module, you can:

- turn a vague complaint into a reproducible query and workload statement;
- distinguish estimated plans from plans with actual execution measurements;
- recognize common scan, join, sort, and aggregate plan nodes;
- connect selectivity and statistics to planner choices;
- design and test a basic single-column, composite, or partial index; and
- decide whether an improvement justifies its write, storage, and maintenance cost.

7.3 "Slow" Is a Symptom, Not a Diagnosis

Begin with a reproducible statement:

- Which exact query and parameter values?
- Which database, schema, and data volume?
- What result grain and row count?
- What elapsed time or service objective?
- Is the time spent executing, waiting for a lock, transferring many rows, or rendering in a client?
- Is the behavior repeatable, and what changed?

A query returning a million rows may be fast for the database and slow for the network or browser. A query waiting on a lock may have a simple plan. Measure the right layer.

The repair desk wants its twenty newest open requests, not a faster version of a different answer. That distinction defines the experiment: keep the query's meaning and data fixed, then change how the DBMS can find the rows. The small twelve-ticket fixture is useful for checking the answer. The separate Week 7 performance fixture supplies 100,000 synthetic rows, including 2,000 open tickets, to make differences in search work visible. It is a teaching workload, not a benchmark of city services or a cloud vendor.

7.4 SQL Describes an Answer; a Plan Describes Work

Chapter 2 used relational algebra to explain what a query means. A physical plan chooses algorithms and access paths that produce that answer. A join could compare rows repeatedly, probe an index, build a hash table, or merge ordered inputs. The correct result should not depend on which suitable algorithm wins. This

separation is part of data independence: an administrator can add an index without requiring every application to rewrite its question.

Cost-based optimization compares estimated alternatives rather than applying "always use an index." The classic System R work by Selinger and colleagues described access-path selection for relational queries in 1979. Its relevance here is the separation of a declarative request from the physical strategy that executes it, not a claim that a modern PostgreSQL plan is identical to System R. IBM's research record

7.5 `EXPLAIN` Shows the Planner's Strategy

Listing 61. SQL

```sql
EXPLAIN
SELECT ticket_id, subject, opened_at
FROM metro_support.tickets
WHERE status = 'open'
ORDER BY opened_at DESC;
```

Plain `EXPLAIN` estimates a plan without executing the planned statement. Use trusted course SQL: planning can still require locks and evaluate eligible constant expressions, so it is not a general sandbox for arbitrary code.

Listing 62. SQL

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_id, subject, opened_at
FROM metro_support.tickets
WHERE status = 'open'
ORDER BY opened_at DESC;
```

`ANALYZE` executes the statement and reports actual timing and rows. On `UPDATE`, `DELETE`, or function calls, that means real side effects unless protected and rolled back. Use it only when execution is safe. `BUFFERS` reports cache and I/O activity available to the statement.

A transaction and rollback can undo ordinary transactional data changes made by an analyzed write, but they do not make external side effects or sequence allocation disappear. In this course, measure SELECT queries in disposable data. Do not benchmark a production DELETE merely because its plan is interesting.

7.6 Read a Plan From the Inside Out

A plan is a tree. Child nodes produce rows for parent nodes.

Common nodes include:

- **Seq Scan:** inspect table pages and test rows. Often correct for a small table or a query returning much of the table.
- **Index Scan:** use an index to find row locations, then access table rows.
- **Index Only Scan:** answer from index entries when required columns and visibility information permit it.
- **Bitmap Index/Heap Scan:** collect many index matches and visit table pages in batches.
- **Nested Loop:** for each row from one input, scan or probe the other input.
- **Hash Join:** build a hash table for one input and match the other, commonly for equality joins.
- **Merge Join:** consume sorted inputs and merge matching keys.

- **Sort:** order rows; may use memory or spill to temporary storage.
- **Aggregate/GroupAggregate/HashAggregate:** reduce input rows into summaries.

Node names are not grades. A sequential scan is not automatically bad, and an index scan is not automatically good.

7.6.1 Follow the Rows Through One Plan

The following excerpt comes from the course fixture on an isolated PostgreSQL 15 server during the September 7, 2026 audit. Timings and estimates will vary on another run. The text is abbreviated to expose the rows and algorithm, not to hide an unfavorable measurement.

Listing 63. Text

```
Limit (actual rows=20 loops=1)
  -> Sort (actual rows=20 loops=1)
       Sort Key: opened_at DESC
       Sort Method: top-N heapsort
       -> Seq Scan on tickets (actual rows=2000 loops=1)
            Filter: status = 'open'
            Rows Removed by Filter: 98000
```

The scan tests 100,000 rows, keeps 2,000, and rejects 98,000. The sort receives 2,000 candidates. A top-N sort can retain a small set of best candidates rather than fully sort every row, but it still has to inspect the input to know which are newest. The parent asks for twenty rows, so only twenty emerge at the top. Seeing `rows=20` at the sort does not mean only twenty candidates were considered.

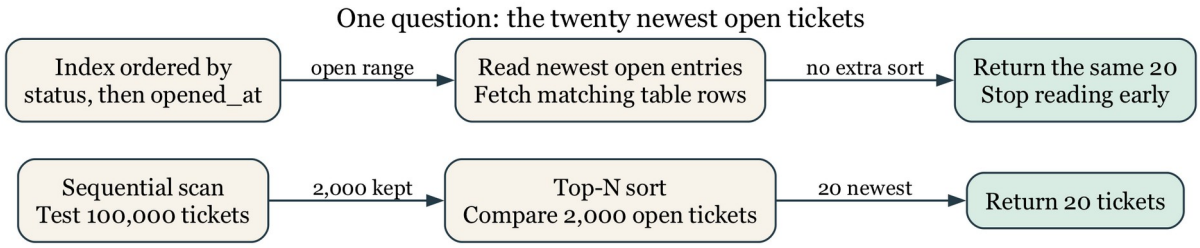

Figure 7.1: The same twenty-row answer can require scanning and comparing many candidates or walking an index in the needed order. Arrows show row flow, not network traffic.

For a node repeated by a nested loop, actual rows and time are reported per loop on average. A child with `rows=3 loops=100` produces about 300 rows across those invocations, not three total. Parent time includes work in its children, so adding every node's time double-counts work. Likewise, a shared buffer **hit** means PostgreSQL found a page in its buffer cache; a **read** does not necessarily mean the physical disk was accessed, because the operating system may cache it. PostgreSQL EXPLAIN guide

7.7 Estimates Drive Choices

The planner estimates row counts and costs using table statistics, data type information, constraints, and configurable assumptions. Compare estimated rows with actual rows in an analyzed plan.

Large mismatches may come from:

- stale statistics;
- correlated columns that basic statistics treat as independent;

- unusual parameter values;
- skewed data;
- expressions without useful statistics; or
- a model that hides important relationships.

`ANALYZE` refreshes statistics:

Listing 64. SQL

```
ANALYZE metro_support.tickets;
```

Do not run maintenance blindly as a ritual. Identify the mismatch and verify the effect.

The fixture's first scan estimated 2,017 open rows and produced 2,000. That is a reasonable estimate, even though the plan still had to read the whole table. After refreshing statistics, another estimate was 1,953. Sampling makes small differences normal. A useful question is whether an error is large enough to favor an unsuitable join or access path, not whether every estimate is exact.

The displayed `cost=startup..total` uses the planner's cost units, not milliseconds. Startup cost estimates work before the first row; total cost estimates work to produce the full node result. LIMIT can favor a path that starts quickly and stops early even if that path's full-result cost is higher.

7.8 Selectivity Explains Many Scan Decisions

Selectivity is the fraction of rows expected to satisfy a condition. An index is often attractive when a query needs a small, identifiable fraction of a large table. If nearly every row has `status = 'open'`, using an index may still require visiting most table pages, making a sequential scan reasonable.

For predicate φ over relation R, define selectivity as:

$$s(\varphi) = \frac{|\sigma_{\varphi}(R)|}{|R|}, 0 \le s(\varphi) \le 1.$$

A predicate matching 500 rows in a 1,000,000-row table has $s(\varphi) = 0.0005$, or $0.05\,\%$. Selectivity alone does not choose a plan, but it helps explain why an index can be attractive for one predicate and wasteful for another.

Here $|R|$ means the number of rows and σ means selection, or keeping rows that match the predicate. The ratio is defined for a nonempty relation; there is no need to divide by zero for an empty table.

The twelve-row class dataset is too small to demonstrate realistic planner tradeoffs. Performance labs create a larger, disposable table with a skewed status distribution. Small examples teach correctness; larger fixtures reveal access paths.

7.9 B-Tree Indexes Support Ordered Comparisons

PostgreSQL's default B-tree index supports equality and range comparisons and can also help ordered retrieval.

A B-tree is a balanced search structure organized into pages. At an internal page, comparisons choose a smaller range of possible keys. Leaf entries lead to matching table rows and support movement through neighboring ordered keys. The tree stays relatively shallow because a page can direct searches to many child pages. This is different from reading every ticket and testing its status.

For a simplified model with roughly b children per internal page and N keys, the number of levels grows on the order of $\log_b N$, rather than N. The notation says how growth behaves; it is not a prediction of milliseconds or an exact PostgreSQL page count. Retrieving many matches still requires processing those matches and possibly visiting many table pages.

Listing 65. SQL

```
CREATE INDEX perf_tickets_status_opened_idx
ON performance_lab.tickets (status, opened_at DESC);
```

A composite index is ordered first by `status`, then by `opened_at` within each status. It may efficiently support this query fragment, which is not a standalone SQL statement:

Listing 66. SQL

```
WHERE status = 'open'
ORDER BY opened_at DESC
LIMIT 20
```

It is less directly suited to a query that filters only `opened_at` without the leading column. Column order should follow real predicates and ordering needs, not a rule such as “most unique first” applied without context.

It is also not correct to say a later column can *never* help without the first. Planner choices depend on the version and distribution; PostgreSQL 18 documents B-tree skip-scan cases. The lab’s core comparison does not depend on that feature. Multicolumn indexes

An index containing `(status, opened_at)` does not also contain every subject text. An ordinary index scan locates entries and fetches the table rows. A covering index can include extra output columns, but larger entries cost space and write work. Even an index-only scan may need table visits to establish row visibility under MVCC. The node name is not a promise of zero heap fetches. Index-only scans

7.10 Partial Indexes Cover a Deliberate Subset

If most tickets are closed but the operational queue reads only active rows, a partial index may be smaller:

Listing 67. SQL

```
CREATE INDEX perf_tickets_active_opened_idx
ON performance_lab.tickets (opened_at DESC)
WHERE status IN ('new', 'open', 'in_progress');
```

The query predicate must imply the index predicate for the planner to use it. Partial indexes add design specificity: if the application’s definition of active changes, the index and queries may need coordinated revision.

7.11 Indexes Have Costs

Each useful index can add:

- storage;
- write work on inserts, updates, and deletes;
- write-ahead log volume;
- backup size or duration;
- cache competition; and
- maintenance and cognitive overhead.

Duplicate, unused, or speculative indexes are not free. Start with a defined workload and baseline measurements.

Not every column update changes every index entry, and PostgreSQL can sometimes use heap-only tuple optimizations when indexed values are unchanged and page conditions permit. The beginner design question remains: does the benefit to the important reads justify maintaining another data structure? A classroom read test does not measure the cost of a busy production write workload.

7.12 Worked Example: Test an Index Hypothesis

Workload: list the 20 newest open tickets in a large operational table.

Start with the linked disposable setup. The preceding index definitions are examples to study, not extra indexes to leave installed before the baseline. Running the setup resets only `performance_lab` and removes its earlier indexes. The setup also refreshes table statistics. Measure the baseline twice before creating an index, then avoid another statistics refresh between measurements. This keeps the proposed access path as the deliberate change. Cache state and background activity may still vary, so it is not a controlled hardware benchmark.

7.12.1 Baseline

Listing 68. SQL

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_id, subject, opened_at
FROM performance_lab.tickets
WHERE status = 'open'
ORDER BY opened_at DESC
LIMIT 20;
```

Record:

- scan node;
- estimated and actual rows at the scan;
- whether a sort occurs;
- planning and execution time; and
- buffer activity.

7.12.2 Hypothesis

An index on `(status, opened_at DESC)` can locate open rows in requested order and may avoid scanning and sorting unrelated rows.

7.12.3 Add the Candidate

Listing 69. SQL

```sql
CREATE INDEX perf_tickets_status_opened_idx
ON performance_lab.tickets (status, opened_at DESC);
```

7.12.4 Remeasure

Run the identical analyzed query twice. Do not change the selected columns, predicate, order, or limit while comparing.

The corresponding audited plan was:

Listing 70. Text

```
Limit (actual rows=20 loops=1)
  -> Index Scan using perf_tickets_status_opened_idx on tickets
       (actual rows=20 loops=1)
       Index Cond: status = 'open'
```

The index supplies matching rows in opening-time order, so no separate sort is needed and the parent stops after twenty. The earlier run recorded 1,191 shared buffer hits; this indexed run recorded 11 hits and four reads. These are local observations of this fixture, not a service-level guarantee. Confirm the same twenty ticket IDs as well as the reduced work. A faster query returning different requests is not an optimization of the original question.

7.12.5 Decide

Keep the index only if the workload benefit is meaningful relative to its cost. A changed node name alone is not enough. On a warm cache and small class fixture, timing can vary; plan shape, rows, sort removal, and buffers may be more stable indicators.

7.13 Separate Query Tuning From Lock Diagnosis

An analyzed plan reports execution after the statement can proceed. If the query waits on another transaction, `pg_stat_activity`, wait events, and `pg_blocking_pids` may be the first useful diagnostic. Do not add an index to solve a transaction left open in a client.

7.14 Common Misconceptions

7.14.1 “Sequential scan means the index is missing”

The table may be small, the condition unselective, the statistics reasonable, or the available index mismatched to the query.

7.14.2 “Lower estimated cost is milliseconds”

Planner cost units are relative estimates, not elapsed-time units.

7.14.3 "`EXPLAIN ANALYZE DELETE` is only an explanation"

`ANALYZE` executes the statement. Use a safe copy and transaction or avoid it.

7.14.4 "One fast run proves the change"

Caching, concurrent load, and measurement noise affect timing. Repeat carefully and compare multiple measurements.

7.15 Study and Practice

The Week 7 guide identifies the assigned reading and one lab submission for each meeting. The questions below are optional self-study, not additional required work.

For a query that filters `assignee_id`, selects active statuses, orders newest first, and returns 25 rows:

1. state the result grain and workload;
2. propose a composite or partial index;
3. predict which plan work it may remove;
4. name three before/after observations; and
5. state one write or maintenance cost.

7.16 Retrieval and Transfer

1. How does `EXPLAIN` differ from `EXPLAIN ANALYZE`?
2. Why might PostgreSQL choose a sequential scan when an index exists?
3. What does a large estimated-versus-actual row mismatch suggest?
4. Why does composite-index column order matter?
5. When can a partial index be useful?
6. Which observations would tell you the query is waiting rather than scanning slowly?

7.17 Further Reading

- PostgreSQL `EXPLAIN`
- PostgreSQL planner statistics
- PostgreSQL index types
- PostgreSQL multicolumn indexes
- PostgreSQL partial indexes

8 A Backup Matters Only When Recovery Works

8.1 Operating Question

If the original database disappears or a harmful change commits, what artifact can recreate the required state, where can it be restored safely, and which checks confirm that the result is usable?

8.2 Learning Outcomes

After this module, you can:

- distinguish high availability, backup, restore, and disaster recovery;
- use recovery point and recovery time objectives to clarify a promise;
- explain logical and physical backup tradeoffs;
- create a free-tier-appropriate PostgreSQL logical backup command;
- restore into a separate target and verify structure, data, and behavior; and
- write a concise runbook with prerequisites, safety boundaries, and verification checks.

8.3 Four Mechanisms Solve Different Problems

At 14:17, an administrator commits a DELETE with the wrong condition. The database is still running, the network is healthy, and other queries work. A server restart will not bring back the deleted rows: from the DBMS's perspective, the deletion was an authorized, committed change. The team needs an earlier recoverable state and a decision about which later valid changes to preserve. This is a recovery problem even though it is not a hardware outage.

- **High availability** reduces service interruption when a component fails.
- **Backup** creates recoverable data outside the active state.
- **Restore** reconstructs data or objects from a backup artifact.
- **Disaster recovery** combines people, systems, procedures, alternate locations, and tested decisions for a serious failure.

A replica can quickly copy an accidental deletion. A backup can exist but be corrupt, incomplete, inaccessible, or impossible to restore within the required time. Redundancy and recovery complement each other.

8.4 RPO and RTO Turn "We Have Backups" Into a Promise

Recovery Point Objective (RPO) is the maximum acceptable data-loss window. A daily export may imply up to roughly one day of lost changes, depending on when failure occurs.

Recovery Time Objective (RTO) is the target time to restore an acceptable service. It includes obtaining credentials, provisioning a target, transferring data, restoring, verifying, and reconnecting consumers.

RPO and RTO are requirements, not properties automatically created by writing them down. Backup frequency, artifact retention, restore speed, and staffing must support them.

If t_f is the failure time and t_r is the timestamp of the newest recoverable state, the realized data-loss window is:

$$\Delta_{\text{loss}} = t_f - t_r.$$

The recovery design meets an RPO target RPO_{max} only when $\Delta_{\text{loss}} \leq RPO_{max}$. If service becomes acceptable again at t_a, the realized recovery duration is $\Delta_{\text{recovery}} = t_a - t_f$ and must be compared with the RTO target. These inequalities turn vague confidence into something that can be measured during a restore drill.

8.4.1 Work Through the Clock Times

Suppose the newest usable export represents the database at 14:00. The harmful deletion occurs at 14:17, is recognized at 14:21, and the team finishes restoration and verification at 14:29. The recoverable-data gap is 17 minutes. The service-recovery interval measured from the failure is 12 minutes, not just the eight minutes spent restoring and checking after detection. An RPO of five minutes and an RTO of ten minutes would both be missed.

The 17-minute gap does not tell us how many records were lost. A quiet period and a burst of thousands of writes can have the same time gap. It also does not mean that every write after 14:00 is necessarily unrecoverable: another supported history source might preserve some. State which sources the calculation assumes. For a running logical export, the state represented is its consistent snapshot, not automatically the wall-clock time when the output file finished writing.

RPO and RTO therefore start a conversation about a service's needs. A historical classroom dataset may tolerate yesterday's state; an active request desk may not. The target must be justified by the consequences of lost work and unavailable service, not copied from a cloud provider's marketing page.

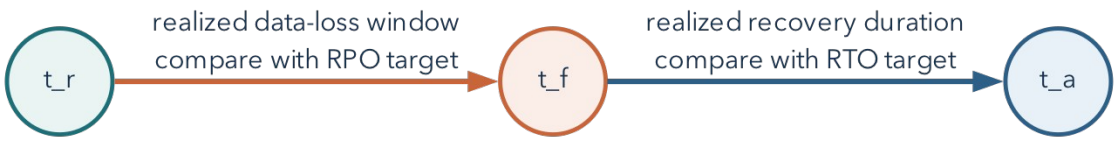

Figure 8.1: RPO constrains the recoverable-data gap before failure; RTO constrains the service-recovery interval after failure.

8.5 Choose the Failure Scope First

Ask what must be recovered:

- one table after an incorrect delete;
- one schema after a migration failure;
- an entire database after project loss;
- credentials and permissions after access corruption;
- a service in a different region; or

- an application-consistent state spanning multiple systems.

The correct artifact and procedure depend on scope. A CSV export of one table may help recover rows but does not preserve keys, constraints, indexes, views, functions, roles, or transactionally consistent relationships by itself.

8.6 Logical and Physical Backups

8.6.1 Logical Backup

Logical tools export SQL objects and data in a form the DBMS can reconstruct. PostgreSQL uses `pg_dump` for one database and `pg_dumpall` for cluster-wide logical content such as roles, within supported permissions.

Benefits include object-level selection and portability across some PostgreSQL versions. Costs can include longer export/restore time and incomplete coverage of cluster-level configuration.

The logical export reconstructs the meaning of objects rather than copying the server's live storage files byte for byte. It can recreate a table, load its rows, and recreate its indexes. A consistent database snapshot keeps related exported rows from representing unrelated moments. By contrast, independently downloading several CSVs while the application changes can produce a child event whose parent ticket is absent from the earlier parent export.

8.6.2 Physical Backup

Physical backups copy database storage in a format tied more closely to the DBMS and version, often combined with write-ahead logs for point-in-time recovery. They can support large-system recovery and precise points but require platform support, storage coordination, and operational expertise.

Managed services may expose snapshots or point-in-time recovery only on selected plans. A recovery design must account for the features actually available on its plan. The required course lab uses free tools.

Write-ahead logging records information needed to recover database changes. A supported physical recovery design can restore a base backup and replay a continuous sequence of retained log records to a selected point. The log is not an ordinary spreadsheet of past business events. Losing required segments or using an incompatible base backup can break the recovery chain. Setting up that production infrastructure is outside the required beginner lab; understanding why a single CSV cannot replace it is not.

8.7 Free-Tier Reality in This Course

Supabase documentation states that projects on the Free plan do not receive the automatic database backups available on paid plans. This course therefore treats logical backup and restore as the required recovery path. Students never need to pay for a backup feature.

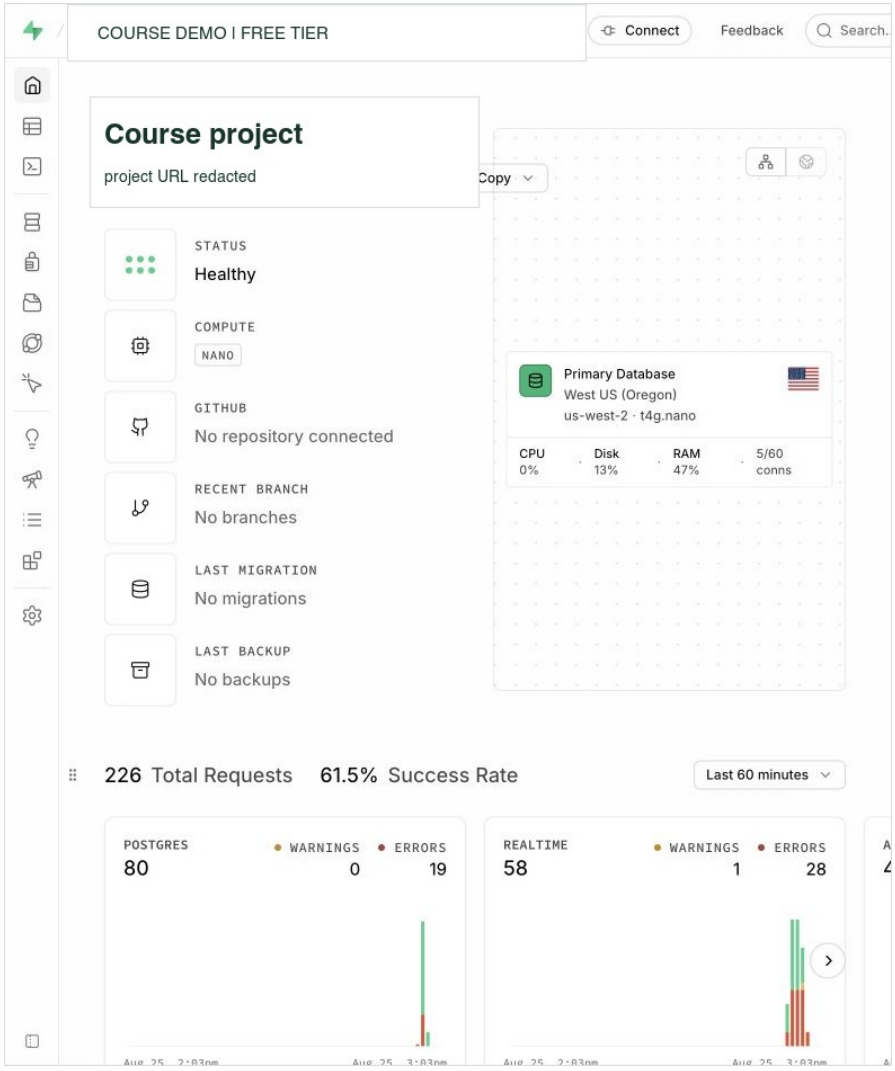

Figure 8.2: A live Supabase Free project reports a healthy database and "No backups" at the same time. Health is present availability; backup is retained recovery history. Project identifiers are redacted. Interface captured August 25, 2026.

The two status fields describe different aspects of the system. **Healthy** describes present availability. **No backups** reports the absence of a retained backup in this displayed project history. Neither field establishes that a separately stored export exists or can be restored. A logical recovery drill tests those additional requirements.

Platform offerings change. Before relying on a backup feature, check the current plan documentation and record that dependency in the recovery plan.

The assigned recovery notebook runs a real PostgreSQL dump and separate restore in disposable local databases. It does not require a paid hosted backup feature. The command-line examples below explain the same tools and provide a reference for a permitted remote source. They are shell commands, not SQL to paste into a web SQL editor.

8.8 Use `pg_dump` Without Publishing a Password

Obtain the source hostname, port, database name, and user from the approved connection panel. Set the non-password shell variables `PGHOST`, `PGPORT`, `PGDATABASE`, and `PGUSER` to those values. `--password` asks

interactively for the password without putting it in the command itself. Do not save a full password-bearing URI in a script or pass it as a visible command-line argument.

For a remote Supabase connection, use a reachable direct endpoint or the IPv4-compatible **session pooler** where needed, and the documented TLS settings. Do not switch off certificate verification to address a network-routing error. Use compatible PostgreSQL client tools: the same major version as the source is a straightforward course choice. An older `pg_dump` cannot dump a newer-major server. A restore to an older-major server is not promised to work.

Custom-format backup of the course schema:

Listing 71. Shell

```
pg_dump \
  --format=custom \
  --schema=metro_support \
  --file=metro_support.dump \
  --host="$PGHOST" --port="$PGPORT" \
  --username="$PGUSER" --dbname="$PGDATABASE" --password
```

Plain SQL backup of one schema:

Listing 72. Shell

```
pg_dump \
  --format=plain \
  --schema=metro_support \
  --no-owner \
  --no-privileges \
  --file=metro_support.sql \
  --host="$PGHOST" --port="$PGPORT" \
  --username="$PGUSER" --dbname="$PGDATABASE" --password
```

For an automated job, use an approved secret store or a restricted PostgreSQL password file rather than inventing a way to echo the password into a command.

In plain SQL output, `--no-owner` and `--no-privileges` omit ownership and grant commands to ease a portable restore. For a custom archive, ownership suppression belongs on `pg_restore`; `pg_dump --no-owner` is ignored for that archive format. The restore below intentionally omits original owners and grants, so a separate reviewed access configuration is necessary before application use. PostgreSQL dump options

`--schema=metro_support` is a deliberate boundary. It is not a backup of the whole Supabase project, its authentication system, file storage, or external services. Objects outside that schema may be dependencies in a real application. A source manifest should say what was included and what was not.

8.9 Inspect the Artifact Before Depending on It

For a custom-format dump:

Listing 73. Shell

```
pg_restore --list metro_support.dump
```

For a plain SQL file, inspect its beginning and end with a text viewer and search for expected table and data statements. Record file size and a cryptographic hash if the artifact will be transferred or retained:

Listing 74. Shell

```
shasum -a 256 metro_support.dump
```

A list or checksum proves properties of the artifact, not that PostgreSQL can successfully restore it.

Treat an untrusted dump as executable input. Current PostgreSQL documentation warns that restoring a dump can execute SQL selected by source superusers. Inspect the source and archive list, restore first into an isolated disposable target with a least-privileged restore role, and never test an untrusted artifact directly against production.

8.10 Restore Into a Separate Target

Never test by overwriting the only copy. Create an empty, disposable database or instructor-approved restore target.

Use separate `RESTORE_PGHOST`, `RESTORE_PGPORT`, `RESTORE_PGUSER`, and `RESTORE_PGDATABASE` variables for the target. Inspect those values before running the restore. A new schema name in the source database is not automatically a separate restore target: the archive recreates the schema names it contains. The course notebook creates a distinct database explicitly.

Custom format:

Listing 75. Shell

```
pg_restore \
  --host="$RESTORE_PGHOST" --port="$RESTORE_PGPORT" \
  --username="$RESTORE_PGUSER" --dbname="$RESTORE_PGDATABASE" \
  --password --single-transaction \
  --no-owner \
  --no-privileges \
  --exit-on-error \
  metro_support.dump
```

Plain SQL:

Listing 76. Shell

```
psql \
  --set=ON_ERROR_STOP=on \
  --host="$RESTORE_PGHOST" --port="$RESTORE_PGPORT" \
  --username="$RESTORE_PGUSER" --dbname="$RESTORE_PGDATABASE" \
  --password --single-transaction \
  --file=metro_support.sql
```

`--exit-on-error` and `ON_ERROR_STOP` make failures visible rather than allowing a long process to appear successful after skipped errors.

For these small course archives, `--single-transaction` also avoids keeping a half-restored set of transactional objects after an error. It is not suitable for every restore mode or every possible script, and it does not clean up old objects already in the target. Begin with the specified empty target; do not add destructive `--clean` options to force a restore over something unfamiliar. PostgreSQL restore options

8.11 Verify Structure, Data, and Behavior

8.11.1 Structure

Listing 77. SQL

```sql
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'metro_support'
ORDER BY table_name;
```

Inspect expected constraints and indexes as well as tables.

8.11.2 Data

Listing 78. SQL

```sql
SELECT 'users' AS object, count(*) FROM metro_support.users
UNION ALL
SELECT 'tickets', count(*) FROM metro_support.tickets
UNION ALL
SELECT 'ticket_events', count(*) FROM metro_support.ticket_events;
```

Counts are necessary but not sufficient. Check a known relationship and a boundary row.

For the full, unchanged Metro Support source fixture, the expected counts are 8 users, 12 tickets, and 21 events. The assigned recovery notebook deliberately uses a smaller fixture: 3 users, 3 tickets, and 5 events. Establish the baseline for the source you actually exported rather than borrowing counts from a different example. In the full fixture, a second check can answer a specific question:

Listing 79. SQL

```sql
SELECT t.ticket_id, t.status, t.assignee_id, u.display_name AS requester
FROM metro_support.tickets AS t
JOIN metro_support.users AS u ON u.user_id = t.requester_id
WHERE t.ticket_id = 1004;
```

The baseline result is ticket 1004, status `new`, a NULL assignee, and requester Jordan Bell. The NULL is expected, not a failed restoration. Compare with the source state you actually backed up if earlier exercises committed changes. Two different databases can have the same row counts, so this identity-and-value check asks a different question from the count query.

8.11.3 Behavior

Run one meaningful report and one expected-failure constraint test inside a transaction. If access configuration is part of the recovery scope, test an allowed and denied operation.

An error alone is insufficient. An invalid status should fail because of the named status constraint, not because the server disconnected or the table name was misspelled. PostgreSQL SQLSTATE `23514` identifies a check-constraint violation. The notebook checks that code and the exact constraint name, then confirms the test row was not retained. It uses a transaction that cannot commit even if a missing constraint unexpectedly allows the insert. This is a small example of testing the failure mechanism rather than accepting any failure as success.

The notebook also compares a ticket's subject and requester between source and target. An incorrect subject can leave every table count unchanged. Conversely, two equal query results can both be wrong if the source

itself was wrong. Compare the result with the intended record in the original fixture as well. Recovery verification and source-data quality answer related but distinct questions.

8.11.4 Application Boundary

If the recovery objective includes application service, use a temporary safe configuration to test a real read and write path. Do not redirect production traffic before verification and approval.

8.12 Worked Example: A Recovery Runbook Entry

Listing 80. Text

```
Purpose: recover the Metro Support schema after an accidental destructive change.

Artifact: custom-format logical dump, created daily by an approved operator.

Prerequisites: pg_dump/pg_restore major version compatible with the server;
temporary source and restore credentials; empty restore target; enough storage.

Safety boundary: never restore over the source database. Never commit URLs.

Procedure: create dump; record exit code, size, and SHA-256; list contents;
restore with --exit-on-error into the separate target.

Verification: compare source and target table definitions; expected row counts;
ticket 1004 and its requester; one report; one rejected foreign-key change.
If security is in scope, restore reviewed roles/grants and test them separately.

Success decision: the required checks pass for this recovery scope. Application
traffic is not redirected until its owner approves the verified target.

Cleanup: revoke temporary credentials and remove the restore target according to
the retention policy.
```

8.13 Safe Migrations and Recovery Are Connected

A backup before a migration is useful only if:

- it includes the necessary scope;
- it can be restored within the decision window;
- the old application remains compatible with the restored state; and
- the team knows whether rollback, forward fix, or restore is safest.

For small changes, transactional DDL and a tested rollback may be faster. For a destructive data transformation, restore may be part of the response. Plan before execution.

8.14 Common Misconceptions

8.14.1 “The dump command returned, so recovery is ready”

Exit status, artifact inspection, separate restore, and verification are still required.

8.14.2 "A CSV is a full database backup"

CSV can preserve selected row values but usually omits schema, constraints, indexes, permissions, and transactionally consistent multi-table state.

8.14.3 "A replica protects against deletion"

Replication can reproduce the deletion. Recovery needs an artifact or history outside the active state and a tested procedure.

8.14.4 "RPO and RTO are the same"

RPO concerns acceptable data loss; RTO concerns acceptable recovery duration.

8.15 Study and Practice

Before Day 1, read through "Free-Tier Reality in This Course" and the verification discussion; use the command explanations as a reference during the notebook. Before Day 2, read the runbook and migration/recovery discussion for the clinic. The assigned lab asks for a real restore and one additional known-record check, not a second extensive checklist. The exercise below is optional self-study.

Write a restore verification checklist for Metro Support with:

1. two structure checks;
2. two data checks;
3. one behavior check;
4. one access check; and
5. one statement of what the checklist does not prove.

8.16 Retrieval and Transfer

1. How does high availability differ from backup?
2. What does an RPO of four hours mean?
3. Why restore into a separate target?
4. What do `--no-owner` and `--no-privileges` trade away?
5. Why is a row count not complete restore verification?
6. Which free-tier constraint changes the required Supabase recovery lab?

8.17 Further Reading

- PostgreSQL backup and restore
- PostgreSQL `pg_dump`
- PostgreSQL `pg_restore`
- Supabase database backups
- Supabase connection guidance

License, Attribution, and Reuse

This volume is a reviewed teaching draft, not an approved institutional OER deposit. The broader textbook remains in development. Vendor documentation, platforms, and educator resources remain their owners' work.

Companion Materials

The accompanying course folder contains eight weekly guides, fourteen individual labs, three notebooks, editable PowerPoints, PDF slide handouts, transcripts, the Metro Support fixture, an optional CISA sample, and the midterm assignment. Open the course README to find the relevant week. In an EPUB reader, download and open the corresponding notebook from that companion folder rather than expecting an EPUB to execute code. No public download address is assumed before the instructor approves publication.